

DATA 260 HOMEWORK 5

Redux Client, MCP Tool Servers, Retry Policy, and an Agent Loop over a Grocery Recall Database

Sharan Lourduraj | SJSU ID: 018169619 | SID4 9619

Section 0 | Personal Configuration and Domain

Value	Result
SID4	9619
PORT_BASE	8619
PREFIX	s9619
SEED	9619
VERIFY_SEED	269619
DOMAIN_ID	3
Assigned domain	Grocery recalls and foodborne-illness outbreaks
Hardware	HP Pavilion laptop, 32 GB RAM, 512 GB storage
Local model	qwen3:8b via Ollama (think=False for the agent runs)

GitHub repository: <https://github.com/Sharanjoo/data260-9619>

Implementation commit reference (hw5 tag): 69fc4b5bf9aa26d35a606ccdfb94857bd2180376

PART 1 | Database CRUD and Redux

Backend CRUD for recall_source and recall_record, the relationship endpoint, and delete protection (409 when a source still has records), then the React client moved to Redux Toolkit.

```
data260-9619 > code > db.py > ...
84 class RecallSource(Base):
85     """HWS Part 1.I required related entity: the reporting/inspection source
86     for a recall (the "author"-equivalent side of the relationship).
87
88     source_name = primary text field, source_region = secondary text field,
89     source_code = required unique field, created_at/updated_at = timestamps.
90     Full CRUD lives in db_routes.py as of HWS (read-only in HW4).
91     """
92
93     __tablename__ = "recall_source"
94
95     id = Column(Integer, primary_keys=True, autoincrement=True)
96     source_name = Column(String(150), nullable=False)
97     source_region = Column(String(100), nullable=False)
98     source_code = Column(String(40), unique=True, nullable=False, index=True)
99     created_at = Column(DateTime, server_default=func.now(), nullable=False)
100     updated_at = Column(DateTime, server_default=func.now(), onupdate=func.now(), nullable=False)
101
102     records = relationship("RecallRecord", back_populates="source")
103
104
105 class RecallRecord(Base):
106     """HWS Part 1.I required primary entity table.
107
108     product_name = primary field, brand_name = secondary field,
109     record_code = required unique field, units_affected = numeric field with
110     a sensible default (0), source_id = FK to the related entity (left
111     nullable -- unchanged from HW4 -- so existing rows created through the
```

```
PS C:\Users\shara\data260-9619>
Sharan Lourduraj | PS C:\Users\shara\data260-9619> docker compose up -d
[+] up 2/2
✓ Container data260-hw4-mysql-s9619 Healthy 0.6s
✓ Container data260-hw3-s9619 Running 0.8s
Sharan Lourduraj | PS C:\Users\shara\data260-9619> python scripts\recorded_run_hw05.py --
usage: recorded_run_hw05.py [-h] ...
recorded_run_hw05.py: error: provide a command after --
Sharan Lourduraj | PS C:\Users\shara\data260-9619> python scripts\test_execute_tool_hw05.py
[PASS] search_recalls: valid input returns ok=true with data
[PASS] search_recalls: empty query is rejected with ok=false
[PASS] recall_detail: valid id returns ok=true with matching data
[PASS] recall_detail: non-existent id is rejected with ok=false
[PASS] source_recall_summary: valid id returns ok=true with aggregate data
[PASS] source_recall_summary: non-existent id is rejected with ok=false
[PASS] execute_tool: unknown tool name returns ok=false instead of raising
[PASS] execute_tool: safety rule blocks search_recalls limit=50 with ok=false
[PASS] run_agent: MockModel that never finishes stops at max_steps

9/9 tests passed
Sharan Lourduraj | PS C:\Users\shara\data260-9619>
Sharan Lourduraj | PS C:\Users\shara\data260-9619> python scripts\demo_retry_scenarios_hw05.py
=== Scenario 1: success on the first attempt ===
ok=True attempts=1 total_latency_ms=0.0
data='fetched on first try'
PASS

=== Scenario 2: fail on attempt 1, succeed on attempt 2 ===
ok=True attempts=2 total_latency_ms=46.0
data='fetched on attempt 2'
PASS

=== Scenario 3: fail on every attempt -> clean error, no crash ===
ok=False attempts=4 total_latency_ms=344.0
errors='attempt 4 failed: simulated permanent failure'
PASS

All three required Part 3.19 scenarios demonstrated successfully.
Sharan Lourduraj | PS C:\Users\shara\data260-9619>
```

```
> aegisops
> civicpulse
> culture-inspire
> visionpulse
> DeviationTrace
> labpair08-handshake
> OrionQueue

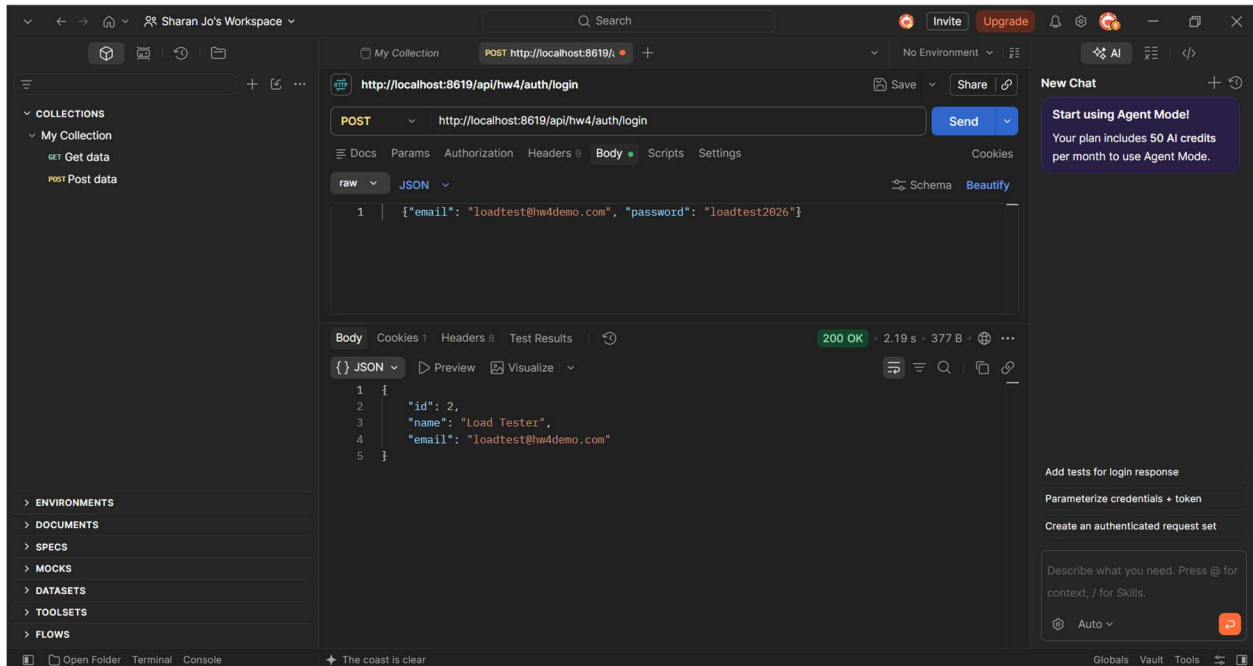
Sharan Lourduraj | PS C:\Users\shara\data260-9619> python scripts/demo_safety_rule_hw05.py
Safety rule: search_recalls limit must be <= 25 per call

=== Allowed call: search_recalls(query='a', limit=10) ===
{
  "ok": true,
  "data": [
    {
      "id": 15007,
      "product_name": "Value Brand Leafy Greens (Lot 00001)",
      "brand_name": "Oakhollow Market",
      "record_code": "REC-015007",
      "units_affected": 0,
      "source_id": 725,
      "source_name": "Local Public Health Unit #125",
      "source_region": "Southern California",
      "created_at": "2026-10-02T05:20:49",
      "updated_at": "2026-10-02T05:20:49"
    },
    {
      "id": 15008,
      "product_name": "Frozen Ice Cream (Lot 00002)",
      "brand_name": "GreenAcre Naturals",
      "record_code": "REC-015008",
      "unit":
    }
  ]
}

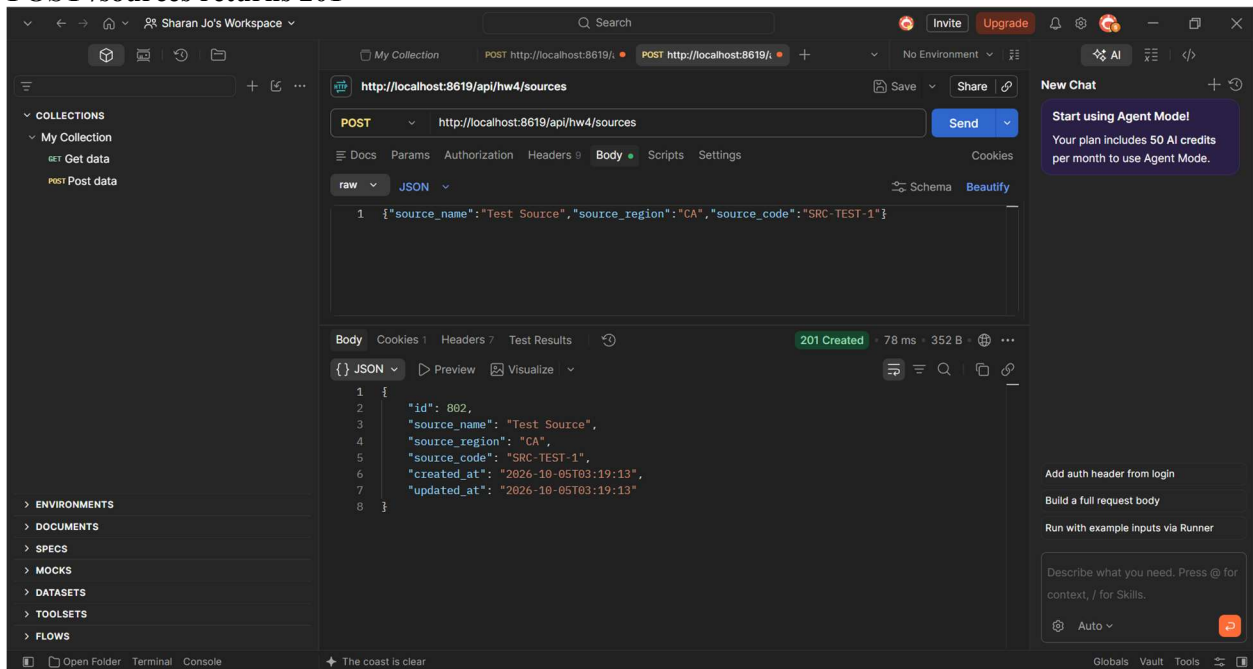
ALLOWED as expected

=== Blocked call: search_recalls(query='a', limit=50) (> 25) ===
{
  "ok": false,
  "data": null,
  "error": "safety rule violated: search_recalls limit=50 exceeds the maximum of 25 allowed per call (prevents bul
--- BLOCKED call: search_recalls(query='a', limit=50) (> 25) ===
{
  "ok": false,
  "data": null,
  "error": "safety rule violated: search_recalls limit=50 exceeds the maximum of 25 allowed per call (prevents bul
k data exfiltration via the agent)"
}
BLOCKED as expected (ok=false, no exception raised)
Sharan Lourduraj | PS C:\Users\shara\data260-9619>
```

Part 1.II - Postman and database



POST /sources returns 201



Duplicate source code returns 409

The screenshot shows the Postman interface with a workspace named "Sharan Jo's Workspace". The active collection is "My Collection". The selected request is a POST to "http://localhost:8619/api/hw4/sources". The request body is a JSON object:

```
{ "source_name": "Test Source", "source_region": "CA", "source_code": "SRC-TEST-1" }
```

. The response is a 409 Conflict with a status bar indicating "75 ms" and "251 B". The response body is a JSON object:

```
{ "detail": "source_code 'SRC-TEST-1' is already in use" }
```

. The right sidebar shows a "New Chat" section with a "Start using Agent Model" button and a "Describe what you need. Press @ for context, / for Skills." input field.

Relationship query: records for source 725

The screenshot shows the Postman interface with a workspace named "Sharan Jo's Workspace". The active collection is "My Collection". The selected request is a GET to "http://localhost:8619/api/hw4/sources/725/records?limit=5". The response is a 200 OK with a status bar indicating "223 ms" and "1.86 KB". The response body is a JSON array of two objects:

```
[ { "id": 15779, "product_name": "Plant-Based Cookie Dough (Lot 08773)", "brand_name": "Amber Grove Provisions", "record_code": "REC-015779", "units_affected": 0, "source_id": 725, "source_name": "Local Public Health Unit #125", "source_region": "Southern California", "source_code": "SRC-08725", "created_at": "2026-10-02T05:20:49", "updated_at": "2026-10-02T05:20:49" }, { "id": 15779, "product_name": "Plant-Based Cookie Dough (Lot 08773)", "brand_name": "Amber Grove Provisions", "record_code": "REC-015779", "units_affected": 0, "source_id": 725, "source_name": "Local Public Health Unit #125", "source_region": "Southern California", "source_code": "SRC-08725", "created_at": "2026-10-02T05:20:49", "updated_at": "2026-10-02T05:20:49" } ]
```

. The right sidebar shows a "New Chat" section with a "Start using Agent Model" button and a "Describe what you need. Press @ for context, / for Skills." input field.

Deleting a source that has records is blocked (409)

The screenshot shows the Postman interface with a DELETE request to `http://localhost:8619/api/hw4/sources/725`. The response is a 409 Conflict with a message: "Cannot delete source 725: 32 recall_record row(s) still reference it".

Request Details:

- Method: DELETE
- URL: `http://localhost:8619/api/hw4/sources/725`
- Headers: No Headers
- Body: Empty

Response Details:

- Status: 409 Conflict
- Time: 68 ms
- Size: 277 B
- Body (JSON):

```
{  "detail": "Cannot delete source 725: 32 recall_record row(s) still reference it"}
```

Right Panel (New Chat):

- Handle source deletion conflict
- Continue in Agent Mode to handle the source deletion conflict.
- Open Source Code
- Think Harder
- Start using Agent Model
- Your plan includes 50 AI credits per month to use Agent Mode.

POST `http://localhost:8619/api/hw4/sources`

The screenshot shows the Postman interface with a POST request to `http://localhost:8619/api/hw4/sources`. The response is a 201 Created with a new source object.

Request Details:

- Method: POST
- URL: `http://localhost:8619/api/hw4/sources`
- Headers: No Headers
- Body (JSON):

```
{  "source_name": "HW5 Demo Source",  "source_region": "NV",  "source_code": "SRC-HW5-DEMO-A"}
```

Response Details:

- Status: 201 Created
- Time: 84 ms
- Size: 360 B
- Body (JSON):

```
{  "id": 806,  "source_name": "HW5 Demo Source",  "source_region": "NV",  "source_code": "SRC-HW5-DEMO-A",  "created_at": "2026-10-05T05:43:05",  "updated_at": "2026-10-05T05:43:05"}
```

Right Panel (New Chat):

- Start using Agent Model
- Your plan includes 50 AI credits per month to use Agent Mode.
- Add pagination/sorting assertions
- Create a reusable request flow
- Run a monitor for regressions

GET http://localhost:8619/api/hw4/sources?limit=5&offset=0

Postman interface showing a GET request to `http://localhost:8619/api/hw4/sources`. The response is a JSON array of 5 source objects, each with fields: `id`, `source_name`, `source_region`, `source_code`, `created_at`, and `updated_at`.

```
18 {
19   "id": 603,
20   "source_name": "State Health Department #003",
21   "source_region": "New England",
22   "source_code": "SRC-00603",
23   "created_at": "2026-10-02T05:20:26",
24   "updated_at": "2026-10-02T05:20:26"
25 },
26 {
27   "id": 604,
28   "source_name": "Consumer Product Safety Bureau #004",
29   "source_region": "South Atlantic",
30   "source_code": "SRC-00604",
31   "created_at": "2026-10-02T05:20:26",
32   "updated_at": "2026-10-02T05:20:26"
33 },
34 {
35   "id": 605,
36   "source_name": "CDC Regional Lab #005",
37   "source_region": "Great Lakes",
38   "source_code": "SRC-00605",
39   "created_at": "2026-10-02T05:20:26",
40   "updated_at": "2026-10-02T05:20:26"
41 }
42 ]
```

GET http://localhost:8619/api/hw4/sources/806

Postman interface showing a GET request to `http://localhost:8619/api/hw4/sources/806`. The response is a single JSON object representing a source with fields: `id`, `source_name`, `source_region`, `source_code`, `created_at`, and `updated_at`.

```
1 {
2   "id": 806,
3   "source_name": "Hw5 Demo Source",
4   "source_region": "NV",
5   "source_code": "SRC-HW5-DEMO-A",
6   "created_at": "2026-10-05T05:43:05",
7   "updated_at": "2026-10-05T05:43:05"
8 }
```

PUT http://localhost:8619/api/hw4/sources/806

The screenshot shows the Postman interface with a PUT request to `http://localhost:8619/api/hw4/sources/806`. The request body is a JSON object: `{ "source_name": "Postman Demo Source Updated", "source_region": "NV" }`. The response status is `200 OK` with a response time of `125 ms` and a body size of `368 B`. The response body is a JSON object: `{ "id": 806, "source_name": "Postman Demo Source Updated", "source_region": "NV", "source_code": "SRC-HWS-DEMO-A", "created_at": "2026-10-05T05:43:05", "updated_at": "2026-10-05T05:45:21" }`. The left sidebar shows the 'My Collection' with 'Get data' and 'Post data' methods. The right sidebar has a 'New Chat' section with a message: 'Start using Agent Model! Your plan includes 50 AI credits per month to use Agent Mode.'

POST http://localhost:8619/api/hw4/records

The screenshot shows the Postman interface with a POST request to `http://localhost:8619/api/hw4/records`. The request body is a JSON object: `{ "product_name": "Postman Test Juice", "brand_name": "Postman Brand", "units_affected": 5, "source_id": 806 }`. The response status is `201 Created` with a response time of `76 ms` and a body size of `503 B`. The response body is a JSON object: `{ "id": 20009, "product_name": "Postman Test Juice", "brand_name": "Postman Brand", "record_code": "REC-83C350AF", "units_affected": 5, "source_id": 806, "source_name": "Postman Demo Source Updated", "source_region": "NV", "source_code": "SRC-HWS-DEMO-A", "created_at": "2026-10-05T05:47:08", "updated_at": "2026-10-05T05:47:08" }`. The left sidebar shows the 'My Collection' with 'Get data' and 'Post data' methods. The right sidebar has a 'New Chat' section with a message: 'Start using Agent Model! Your plan includes 50 AI credits per month to use Agent Mode.'

DELETE http://localhost:8619/api/hw4/records/20009

The screenshot shows the Postman interface with a DELETE request to `http://localhost:8619/api/hw4/records/20009`. The request is successful, returning a `200 OK` status with a response time of 111 ms and a body size of 224 B. The response body is a JSON object: `{ "message": "Record 20009 deleted" }`. The interface includes a sidebar with collections, a top bar with search and navigation, and a right sidebar with a 'New Chat' section.

DELETE http://localhost:8619/api/hw4/sources/806

The screenshot shows the Postman interface with a DELETE request to `http://localhost:8619/api/hw4/sources/806`. The request is successful, returning a `200 OK` status with a response time of 75 ms and a body size of 222 B. The response body is a JSON object: `{ "message": "Source 806 deleted" }`. The interface includes a sidebar with collections, a top bar with search and navigation, and a right sidebar with a 'New Chat' section.

GET http://localhost:8619/api/hw4/sources/999999

The screenshot shows the Postman interface with a GET request to `http://localhost:8619/api/hw4/sources/999999`. The request returns a `404 Not Found` status with a response time of 26 ms and a body size of 233 B. The response body is a JSON object: `{ "detail": "Source 999999 not found" }`. The interface includes a sidebar with collections, a top bar with search and navigation, and a right sidebar with a 'New Chat' section.

POST http://localhost:8619/api/hw4/sources

POST http://localhost:8619/api/hw4/sources

Body

```
{
  "source_name": "Bad Source",
  "source_region": "CA",
  "source_code": "a b!"
}
```

422 Unprocessable Entity (WebDAV) (RFC 4918) · 32 ms · 402 B

Body

```
{
  "detail": [
    {
      "type": "string_pattern_mismatch",
      "loc": [
        "body",
        "source_code"
      ],
      "msg": "String should match pattern '^([A-Za-z0-9_-]{3,40}$'",
      "input": "a b!",
      "ctx": {
        "pattern": "^([A-Za-z0-9_-]{3,40}$"
      }
    }
  ]
}
```

Fix request body JSON

Add auth to POST

Chain create→list→delete

Describe what you need. Press @ for context, / for Skills.

Auto

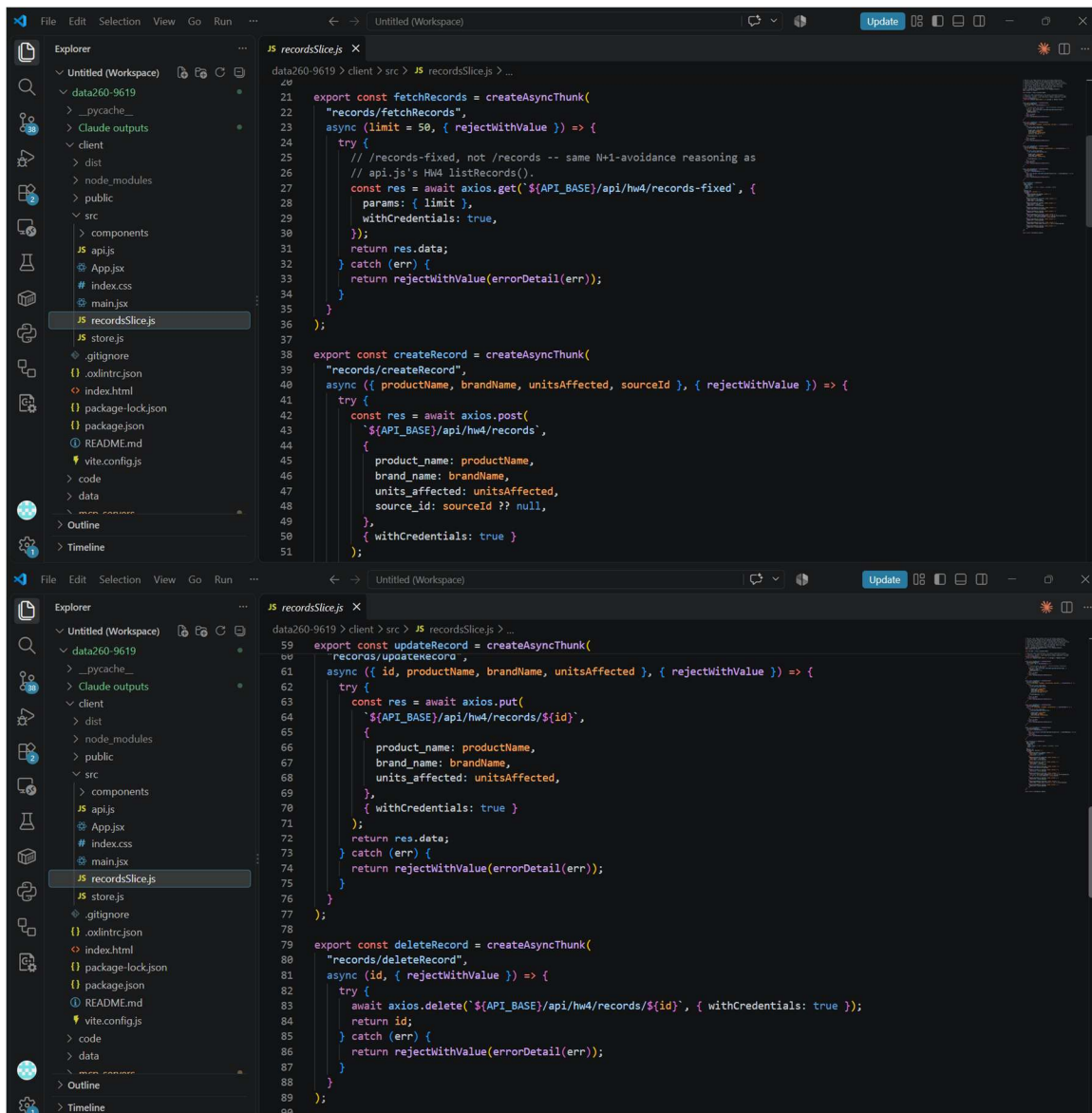
recall source rows in MySQL

```
Sharan Lourduraj | PS C:\Users\shara\data260-9619> docker exec -it data260-hw4-mysql-s9619 mysql -uuhw_user -p4_pass_9619 s9619_rel -e "SELECT id,source_code,source_name FROM recall_source ORDER BY id DESC LIMIT 5;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+-----+-----+
| id | source_code | source_name |
+-----+-----+-----+
| 882 | SRC-TEST-1 | Test Source |
| 888 | SRC-88888 | County Health Inspector #280 |
| 799 | SRC-88799 | County Health Inspector #199 |
| 798 | SRC-88798 | USDA FSTS Field Office #198 |
| 797 | SRC-88797 | Consumer Product Safety Bureau #197 |
+-----+-----+-----+
Sharan Lourduraj | PS C:\Users\shara\data260-9619>
```

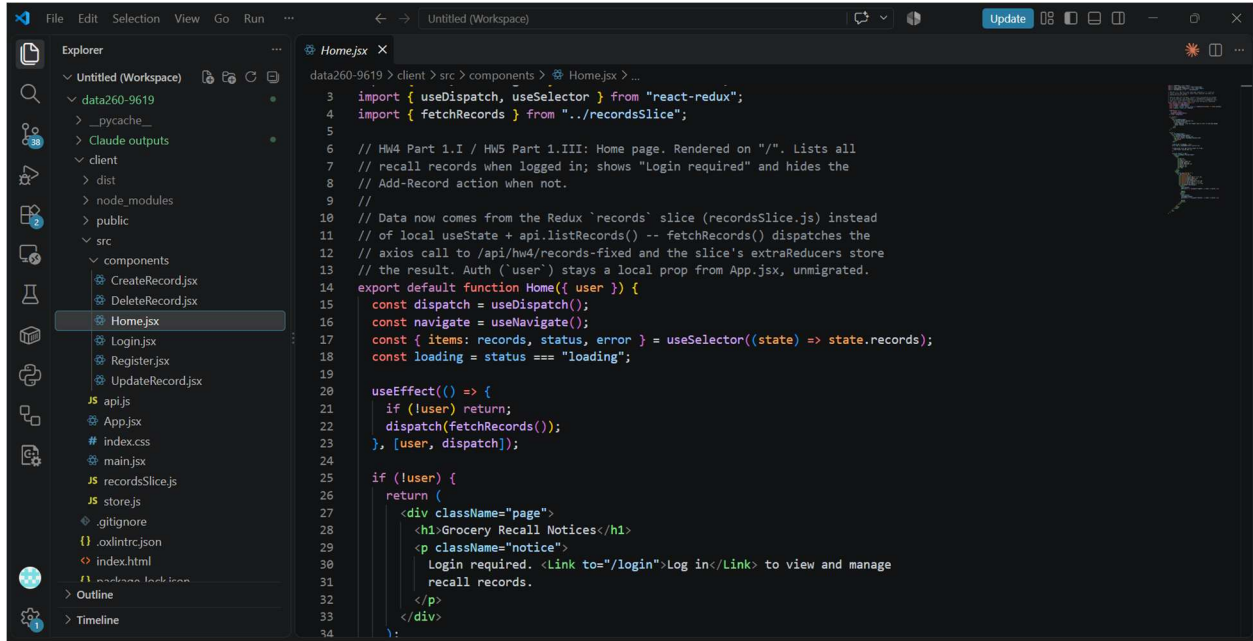
Part 1.III - Redux

Store and async thunks

```
data260-9619 > client > src > JS store.js > ...
1 // HW4 Part 1.III: Redux store. Single "records" slice for the primary
2 // domain entity (recall_record) -- auth/session stays local useState in
3 // App.jsx per the migration scope noted in recordsSlice.js.
4 import { configureStore } from "@reduxjs/toolkit";
5 import recordsReducer from "../recordsSlice";
6
7 export const store = configureStore({
8   reducer: {
9     records: recordsReducer,
10   },
11 });
12
```

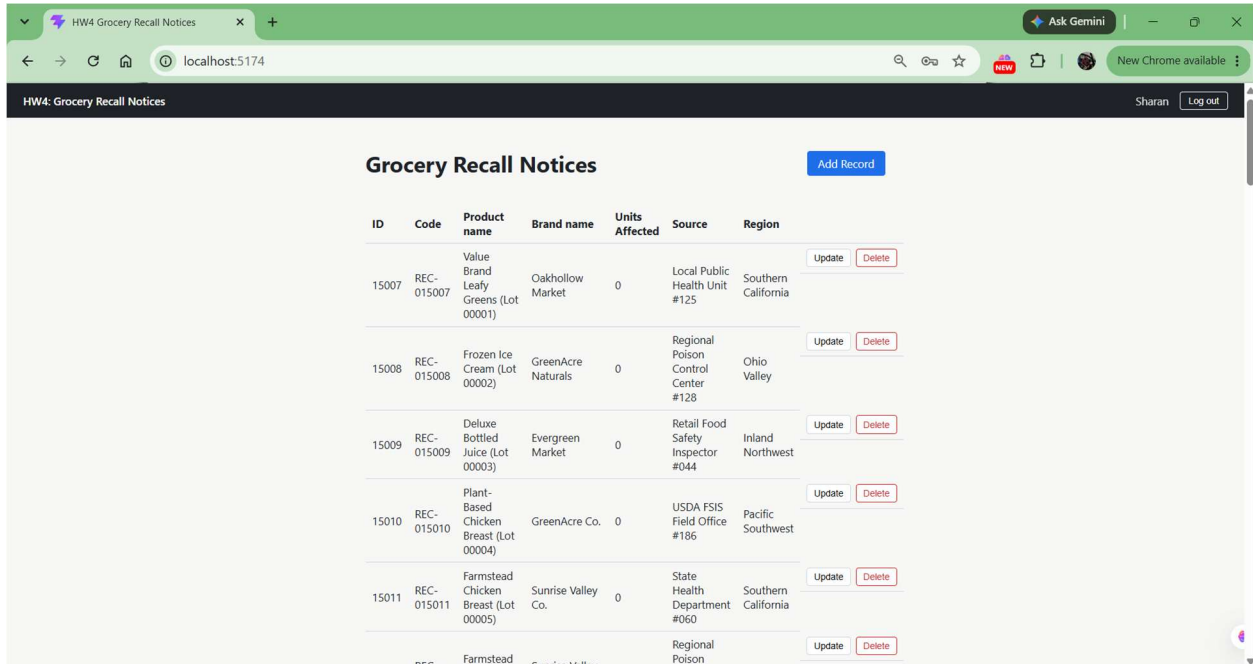


Home: Redux code

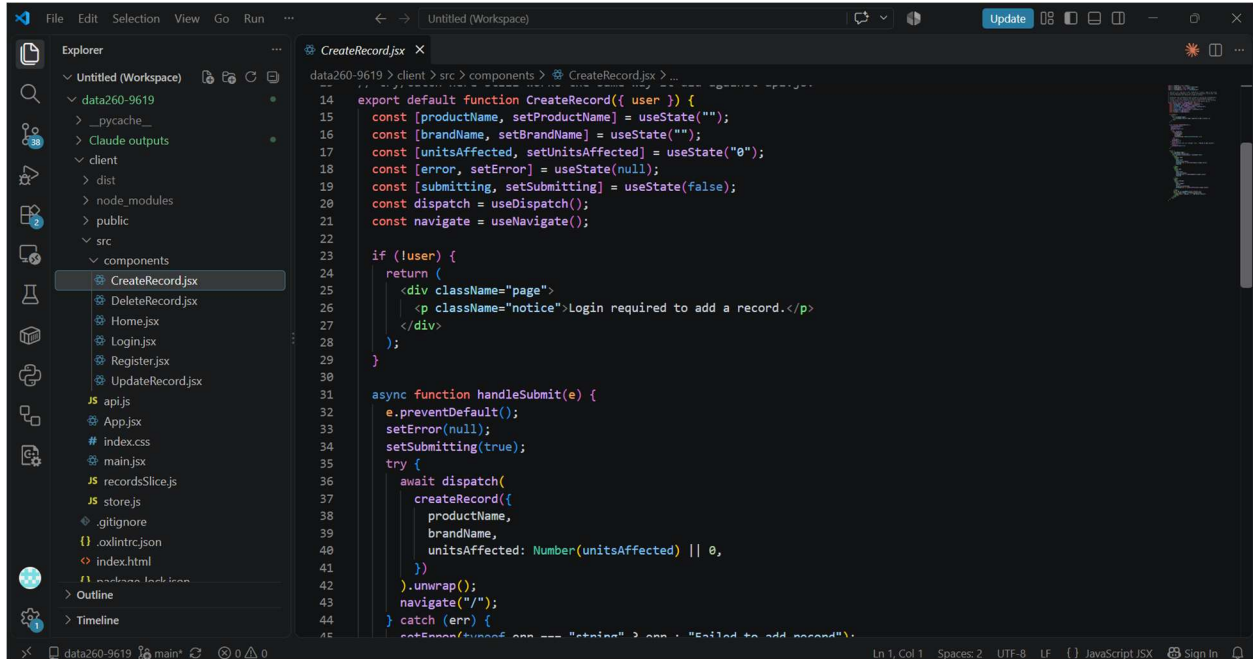


```
3 import { useDispatch, useSelector } from "react-redux";
4 import { fetchRecords } from "../recordsSlice";
5
6 // HW4 Part 1.I / HW5 Part 1.III: Home page. Rendered on "/". Lists all
7 // recall records when logged in; shows "Login required" and hides the
8 // Add-Record action when not.
9 //
10 // Data now comes from the Redux `records` slice (recordsSlice.js) instead
11 // of local useState + api.listRecords() -- fetchRecords() dispatches the
12 // axios call to /api/hw4/records-fixed and the slice's extraReducers store
13 // the result. Auth (`user`) stays a local prop from App.jsx, unmigrated.
14 export default function Home({ user }) {
15   const dispatch = useDispatch();
16   const navigate = useNavigate();
17   const { items: records, status, error } = useSelector((state) => state.records);
18   const loading = status === "loading";
19
20   useEffect(() => {
21     if (!user) return;
22     dispatch(fetchRecords());
23   }, [user, dispatch]);
24
25   if (!user) {
26     return (
27       <div className="page">
28         <h1>Grocery Recall Notices</h1>
29         <p className="notice">
30           Login required. <Link to="/login">Log in</Link> to view and manage
31           recall records.
32         </p>
33       </div>
34     );
35   }
```

Home: UI

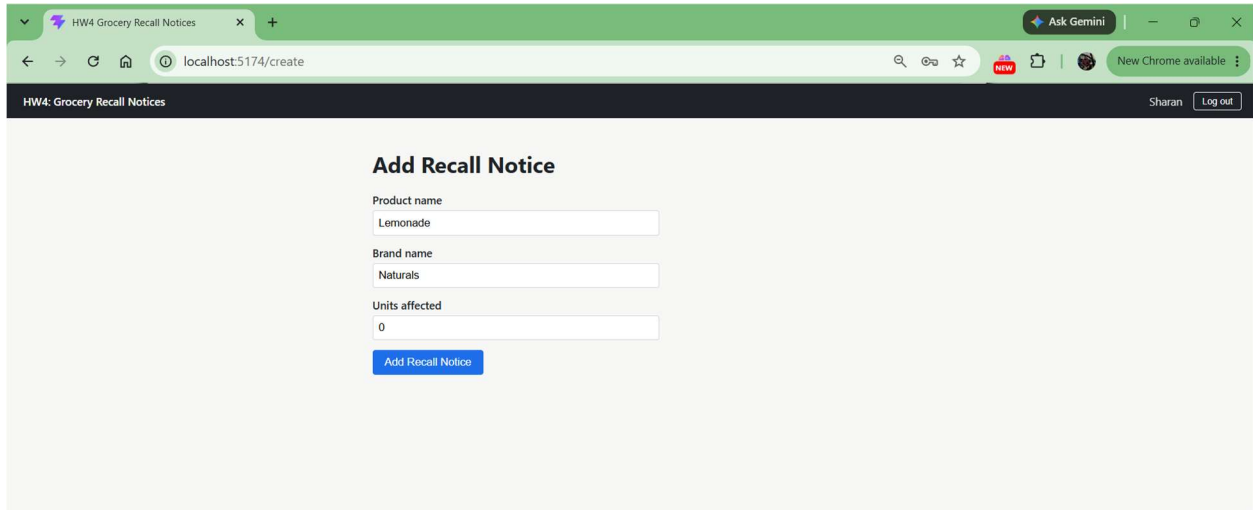


Create: Redux code

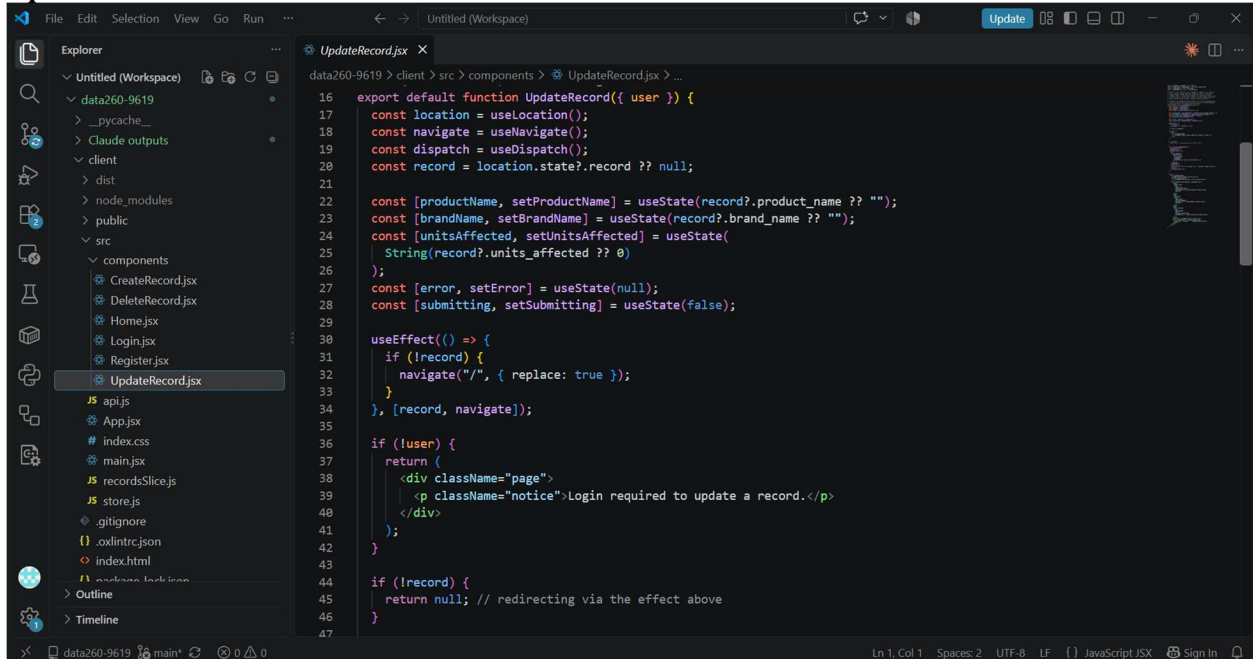


```
14 export default function CreateRecord({ user }) {
15   const [productName, setProductName] = useState("");
16   const [brandName, setBrandName] = useState("");
17   const [unitsAffected, setUnitsAffected] = useState("0");
18   const [error, setError] = useState(null);
19   const [submitting, setSubmitting] = useState(false);
20   const dispatch = useDispatch();
21   const navigate = useNavigate();
22
23   if (!user) {
24     return (
25       <div className="page">
26         <p className="notice">Login required to add a record.</p>
27       </div>
28     );
29   }
30
31   async function handleSubmit(e) {
32     e.preventDefault();
33     setError(null);
34     setSubmitting(true);
35     try {
36       await dispatch(
37         createRecord({
38           productName,
39           brandName,
40           unitsAffected: Number(unitsAffected) || 0,
41         })
42       ).unwrap();
43       navigate("/");
44     } catch (err) {
45       setError(err.message || "Failed to add record.");
46     }
47   }
48 }
```

Create: UI

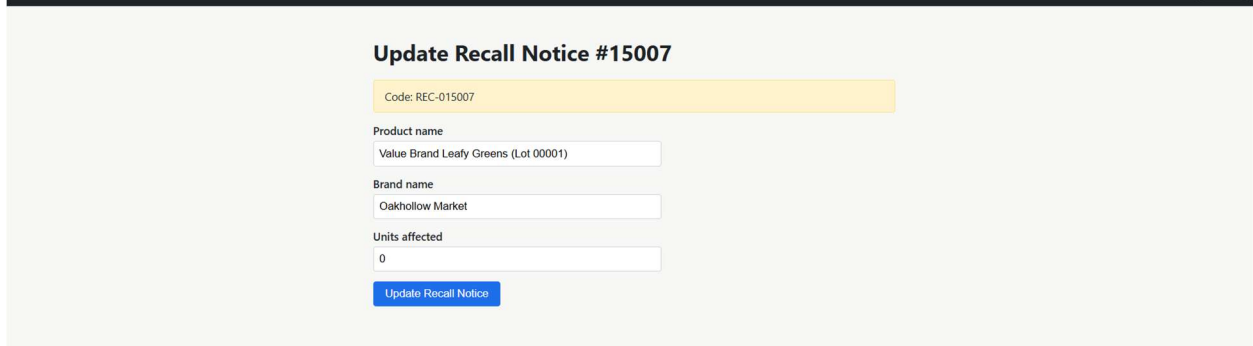
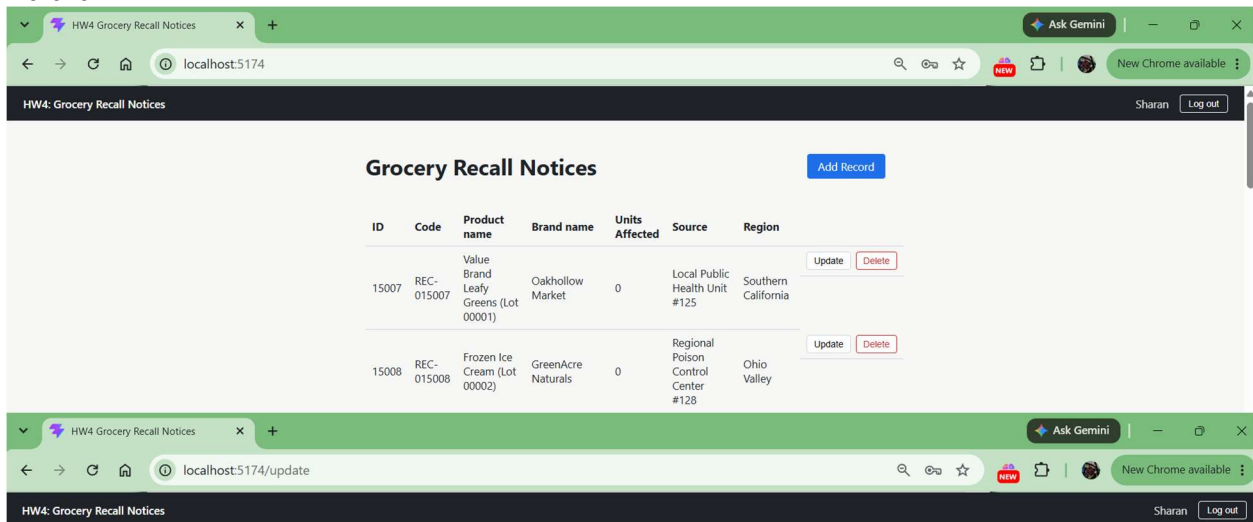


Update: Redux code



```
16 export default function UpdateRecord({ user }) {
17   const location = useLocation();
18   const navigate = useNavigate();
19   const dispatch = useDispatch();
20   const record = location.state?.record ?? null;
21
22   const [productName, setProductName] = useState(record?.product_name ?? "");
23   const [brandName, setBrandName] = useState(record?.brand_name ?? "");
24   const [unitsAffected, setUnitsAffected] = useState(
25     String(record?.units_affected ?? 0)
26   );
27   const [error, setError] = useState(null);
28   const [submitting, setSubmitting] = useState(false);
29
30   useEffect(() => {
31     if (!record) {
32       navigate("/", { replace: true });
33     }
34   }, [record, navigate]);
35
36   if (!user) {
37     return (
38       <div className="page">
39         <p className="notice">Login required to update a record.</p>
40       </div>
41     );
42   }
43
44   if (!record) {
45     return null; // redirecting via the effect above
46   }
47 }
```

Update: UI Before



After

HW4: Grocery Recall Notices

Update Recall Notice #15007

Code: REC-015007

Product name
Value Brand Leafy Greens (Lot 00001)

Brand name
Farmer's Market

Units affected
0

Update Recall Notice

HW4: Grocery Recall Notices

Grocery Recall Notices

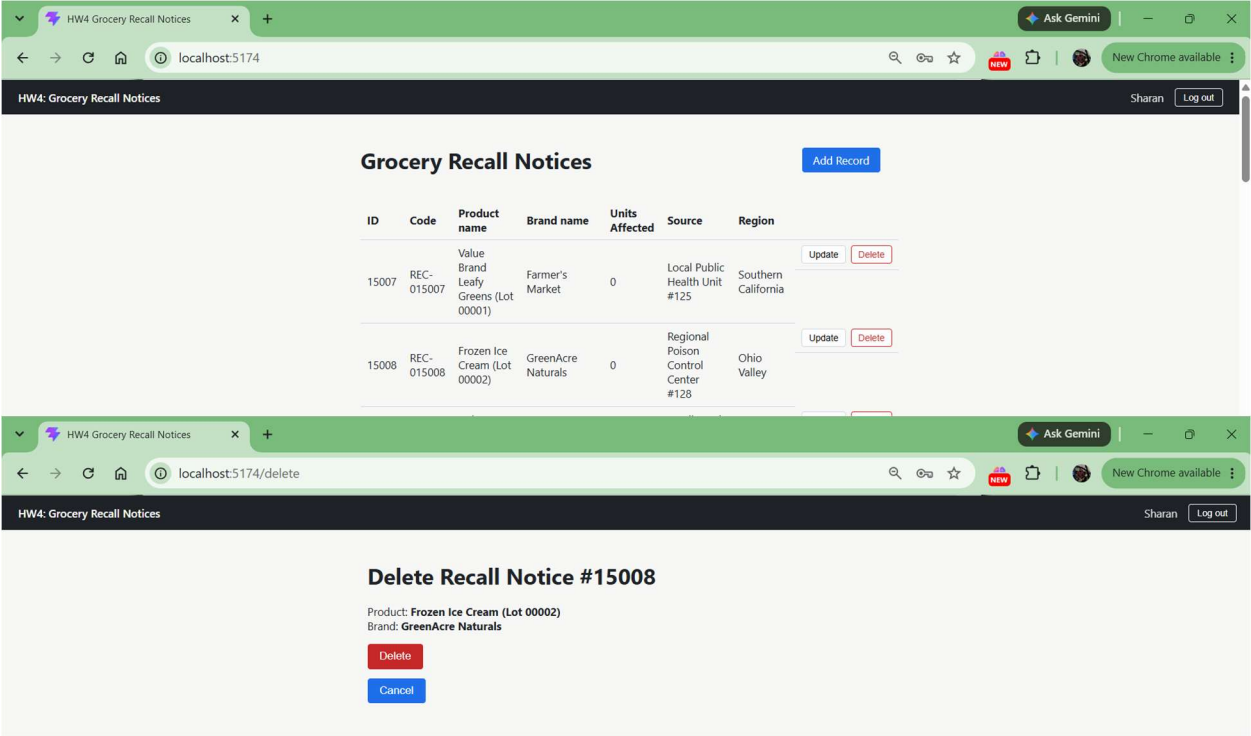
Add Record

ID	Code	Product name	Brand name	Units Affected	Source	Region	
15007	REC-015007	Value Brand Leafy Greens (Lot 00001)	Farmer's Market	0	Local Public Health Unit #125	Southern California	Update Delete
15008	REC-015008	Frozen Ice Cream (Lot 00002)	GreenAcre Naturals	0	Regional Poison Control Center #128	Ohio Valley	Update Delete

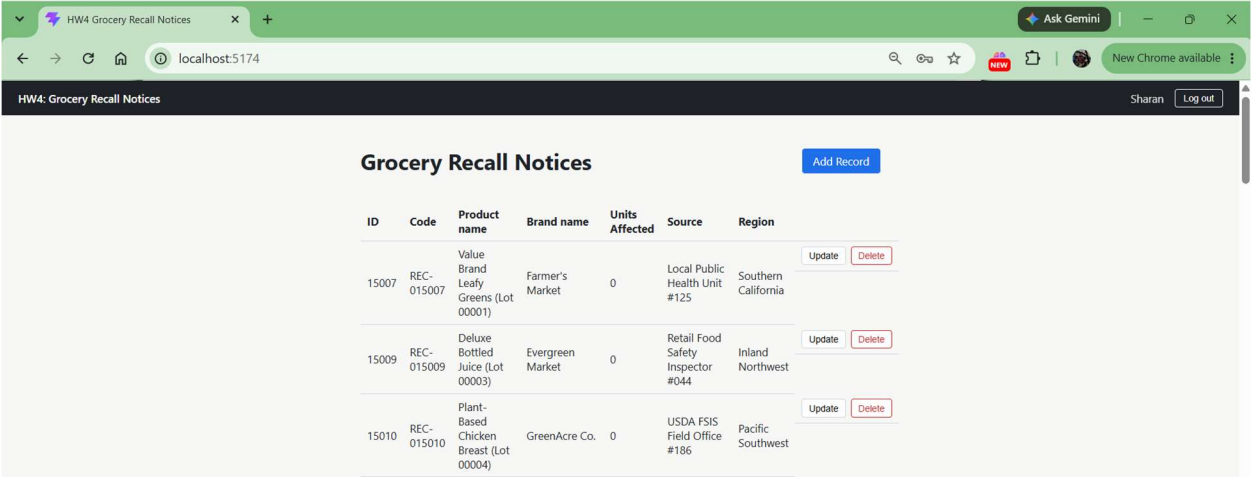
Delete: Redux code

```
14 export default function DeleteRecord({ user }) {
15   // console.log('DeleteRecord', user);
16   return (
17     <div className="page">
18       <p className="notice">Login required to delete a record.</p>
19     </div>
20   );
21 }
22
23 if (!record) {
24   return null; // redirecting via the effect above
25 }
26
27 async function handleDelete() {
28   setError(null);
29   setSubmitting(true);
30   try {
31     await dispatch(deleteRecord(record.id)).unwrap();
32     navigate("/");
33   } catch (err) {
34     setError(typeof err === "string" ? err : "Failed to delete record");
35     setSubmitting(false);
36   }
37 }
38
39 return (
40   <div className="page">
41     <h1>Delete Recall Notice #{record.id}</h1>
42     <p>
43       Product: <strong>{record.product_name}</strong>
44       <br />
45       Brand: <strong>{record.brand_name}</strong>
46     </p>
47   </div>
48 );
```

Delete: UI



After



PART 2 | MCP Servers

Part 2A - MealDB tutorial server (MCP Inspector)

The image displays a development environment with a code editor and a web browser. The code editor shows the implementation of the `search_meals_by_name` function in `meals_server.py`. The function takes a `query` string and a `limit` integer, returning a list of meal objects or an error message. The web browser shows the MCP Inspector interface, which lists available tools and displays a list of messages.

```
115
116 @mcp.tool()
117 def search_meals_by_name(query: str, limit: int = 5) -> dict:
118     """Search TheMealDB for meals by name. Returns up to `limit` (1-25)
119     {id, name, area, category, thumb} objects."""
120     if not isinstance(query, str) or not query.strip():
121         return {"meals": [], "message": "no matches: query must be non-empty"}
122     try:
123         limit = max(1, min(int(limit), 25))
124     except (TypeError, ValueError):
125         raise RuntimeError("limit must be an integer")
126
127     data = _get("search.php", {"s": query.strip()})
128     meals = data.get("meals")
129     if not meals:
130         return {"meals": [], "message": "no matches"}
131     return {"meals": [_meal_summary(m) for m in meals[:limit]], "message": None}
132
```

The MCP Inspector web interface shows the following tools:

- search_meals_by_name
- meals_by_ingredient
- random_meal
- meal_details

The Messages section shows the following log entries:

- 03:37:33 CLIENT -> SER. 1012ms
- 03:37:33 CLIENT -> SER. 929ms
- 03:37:33 CLIENT -> SER. 774ms

search meals by name

The screenshot shows the MCP Inspector interface with the 'search_meals_by_name' tool selected. The query is 'Chicken' and the limit is 3. The results are displayed in the center panel, showing three meal cards with details like name, area, category, and thumb URL.

Tools

- search_meals_by_name
- meals_by_ingredient
- random_meal
- meal_details

search_meals_by_name

Search TheMealDB for meals by name. Returns up to "limit" (1-25) {id, name, area, category, thumb} objects.

Query: Chicken

Limit: 3

Execute Tool

Results

```
1 {
2   "meals": [
3     {
4       "id": "52795",
5       "name": "Chicken Handi",
6       "area": "India",
7       "category": "Chicken",
8       "thumb": "https://www.themealdb.com/images/media/meals/...",
9     },
10    {
11      "id": "53358",
12      "name": "Chicken Mandi",
13      "area": "India",
14      "category": "Chicken",
15      "thumb": "https://www.themealdb.com/images/media/meals/...",
16    },
17    {
18      "id": "53110",
19      "name": "Sticky Chicken",
20      "area": "Australian",
21      "category": "Chicken",
22      "thumb": "https://www.themealdb.com/images/media/meals/...",
23    }
24  ],
25  "message": null
26 }
```

meals by ingredient

```
134 @mcp.tool()
135 def meals_by_ingredient(ingredient: str, limit: int = 12) -> dict:
136     """Filter TheMealDB meals by main ingredient. Returns up to 'limit'
137     {id, name, thumb} cards."""
138     if not isinstance(ingredient, str) or not ingredient.strip():
139         return {"meals": [], "message": "no matches: ingredient must be non-empty"}
140     try:
141         limit = max(1, int(limit))
142     except (TypeError, ValueError):
143         raise RuntimeError("limit must be an integer")
144
145     data = _get("filter.php", {"i": ingredient.strip()})
146     meals = data.get("meals")
147     if not meals:
148         return {"meals": [], "message": "no matches"}
149     return {"meals": [_meal_card(m) for m in meals[:limit]], "message": None}
150
151
```

The screenshot displays the 'meals' application interface. On the left, a 'Tools' panel lists available tools: 'search_meals_by_name', 'meals_by_ingredient' (selected), 'random_meal', and 'meal_details'. The main panel shows the 'meals_by_ingredient' tool configuration. The 'Ingredient' field is set to 'chicken_breast' and the 'Limit' is set to 20. An 'Execute Tool' button is visible. On the right, a 'Messages' panel shows a log of interactions, including a 'CLIENT -> SERVER' message at 03:37:33 and a 'TOOLS/CALL' message for 'search_meals_by_ingredient'.

Below the application interface, a browser window shows the URL: `127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=895ad82d46b39cf82ec299b7d793566794dba5498e4b7c42987b8cb1868fb112`. The browser also shows a 'New Chrome available' notification.

The application interface is also shown with the 'Results' panel open, displaying a list of meal details for the search results. The results are as follows:

ID	Name	Thumb
52820	Katsu Chicken curry	https://www.themeadb.com/images/media/meals/
53192	Panang chicken curry (kaeng panang gai)	https://www.themeadb.com/images/media/meals/
52933	Rappie Pie	https://www.themeadb.com/images/media/meals/
53206	Thai chicken cakes with sweet chilli sauce	https://www.themeadb.com/images/media/meals/
53241	Vietnamese veg parcels	https://www.themeadb.com/images/media/meals/

The 'Messages' panel also shows a log of interactions, including a 'CLIENT -> SERVER' message at 03:39:06 and a 'TOOLS/CALL' message for 'meals_by_ingr'.

At the bottom of the application interface, the version 'v2.9.0' and the copyright notice 'Copyright © Model Context Protocol a Series of LF Projects, LLC.' are displayed.

random_meal

```

151
152 @mcp.tool()
153 def random_meal() -> dict:
154     """Return one random meal with full recipe details (same shape as meal_details)."""
155     data = _get("random.php", {})
156     meals = data.get("meals")
157     if not meals:
158         return {"meal": None, "message": "no matches"}
159     return {"meal": _meal_full(meals[0]), "message": None}
160
161

```

127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=895ad82d46b39cf82ec299b7d793566794dba5498e4b7c42987b8cb1868fb112

meals Servers Tools Prompts Resources

Tools

Search tools...

☐ Paginated

search_meals_by_name

meals_by_ingredient

random_meal

meal_details

random_meal

Return one random meal with full recipe details (same shape as meal_details).

Edit as JSON ☐

Execute Tool

Protocol Console Search...

Messages LEGACY

Newest First ☐ Clear

Export ☐

03:39:06 CLIENT → SERVER 1263n

TOOLS/CALL meals_by_ingr ↺ ⌵

03:38:07 CLIENT → SERVER 2085n

TOOLS/CALL search_meals_ ↺ ⌵

127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=895ad82d46b39cf82ec299b7d793566794dba5498e4b7c42987b8cb1868fb112

meals Servers Tools Prompts Resources

Tools

Search tools...

☐ Paginated

search_meals_by_name

meals_by_ingredient

random_meal

meal_details

Results

```

1 - {
2 -   "meal": {
3 -     "id": "53533",
4 -     "name": "Breaded Steak Recipe (Lomo de Res Apanado)",
5 -     "category": "Beef",
6 -     "area": null,
7 -     "instructions": "Place each piece of steak in between two
8 -     "image": "https://www.themealdb.com/images/media/meals/ln
9 -     "source": "https://www.mycolumbianrecipes.com/breaded-ste
10 -    "youtube": "",
11 -    "ingredients": [
12 -      {
13 -        "name": "Beef",
14 -        "measure": "2 lbs"
15 -      },
16 -      {
17 -        "name": "Alinos Sauce",
18 -        "measure": "1 cup"
19 -      },
20 -      {
21 -        "name": "All purpose flour",
22 -        "measure": "1 cup"
23 -      },
24 -      {
25 -        "name": "Breadcrumbs",
26 -        "measure": "2 cups"
27 -      },
28 -      {
29 -        "name": "Ground Coriander"
30 -      }
31 -    ]
32 -   }
33 - }

```

Protocol Console Search...

Messages LEGACY

Newest First ☐ Clear

Export ☐

03:39:43 CLIENT → SERVER 1309n

TOOLS/CALL random_meal ↺ ⌵

03:39:06 CLIENT → SERVER 1263n

TOOLS/CALL meals_by_ingr ↺ ⌵

03:38:07 CLIENT → SERVER 2085n

meal details

```

161
162 @mcp.tool()
163 def meal_details(id: str) -> dict:
164     """Look up one TheMealDB meal by id and return its full recipe details."""
165     if id is None or str(id).strip() == "":
166         return {"meal": None, "message": "no matches: id must be non-empty"}
167     data = _get("lookup.php", {"i": str(id).strip()})
168     meals = data.get("meals")
169     if not meals:
170         return {"meal": None, "message": "no matches"}
171     return {"meal": _meal_full(meals[0]), "message": None}
172
173

```

meals

ServersToolsPromptsResources

Tools

Search tools...

Paginated

search_meals_by_name

meals_by_ingredient

random_meal

meal_details

meal_details

Look up one TheMealDB meal by id and return its full recipe details.

Id *
52772

Execute Tool

ProtocolConsole

Search...

Messages

LEGACY

Newest First

Clear

Export

03:39:43 CLIENT → SERVER 1309n

TOOLS/CALL random_meal

03:39:06 CLIENT → SERVER 1263n

TOOLS/CALL meals_by_ingr

meals

ServersToolsPromptsResources

Tools

Search tools...

Paginated

search_meals_by_name

meals_by_ingredient

random_meal

meal_details

Results

```
1 {
2   "meal": {
3     "id": "52772",
4     "name": "Teriyaki Chicken Casserole",
5     "category": "Chicken",
6     "area": "Japanese",
7     "instructions": "Preheat oven to 350° F. Spray a 9x13-inc
8     "image": "https://www.themealdb.com/images/media/meals/vv
9     "source": null,
10    "youtube": "https://www.youtube.com/watch?v=4aZr5hzXP_s",
11    "ingredients": [
12      {
13        "name": "soy sauce",
14        "measure": "3/4 cup"
15      },
16      {
17        "name": "water",
18        "measure": "1/2 cup"
19      },
20      {
21        "name": "brown sugar",
22        "measure": "1/4 cup"
23      },
24      {
25        "name": "ground ginger",
26        "measure": "1/2 teaspoon"
27      },
28      {
29        "name": "minced garlic"
```

ProtocolConsole

Search...

Messages

LEGACY

Newest First

Clear

Export

03:40:18 CLIENT → SERVER 1283n

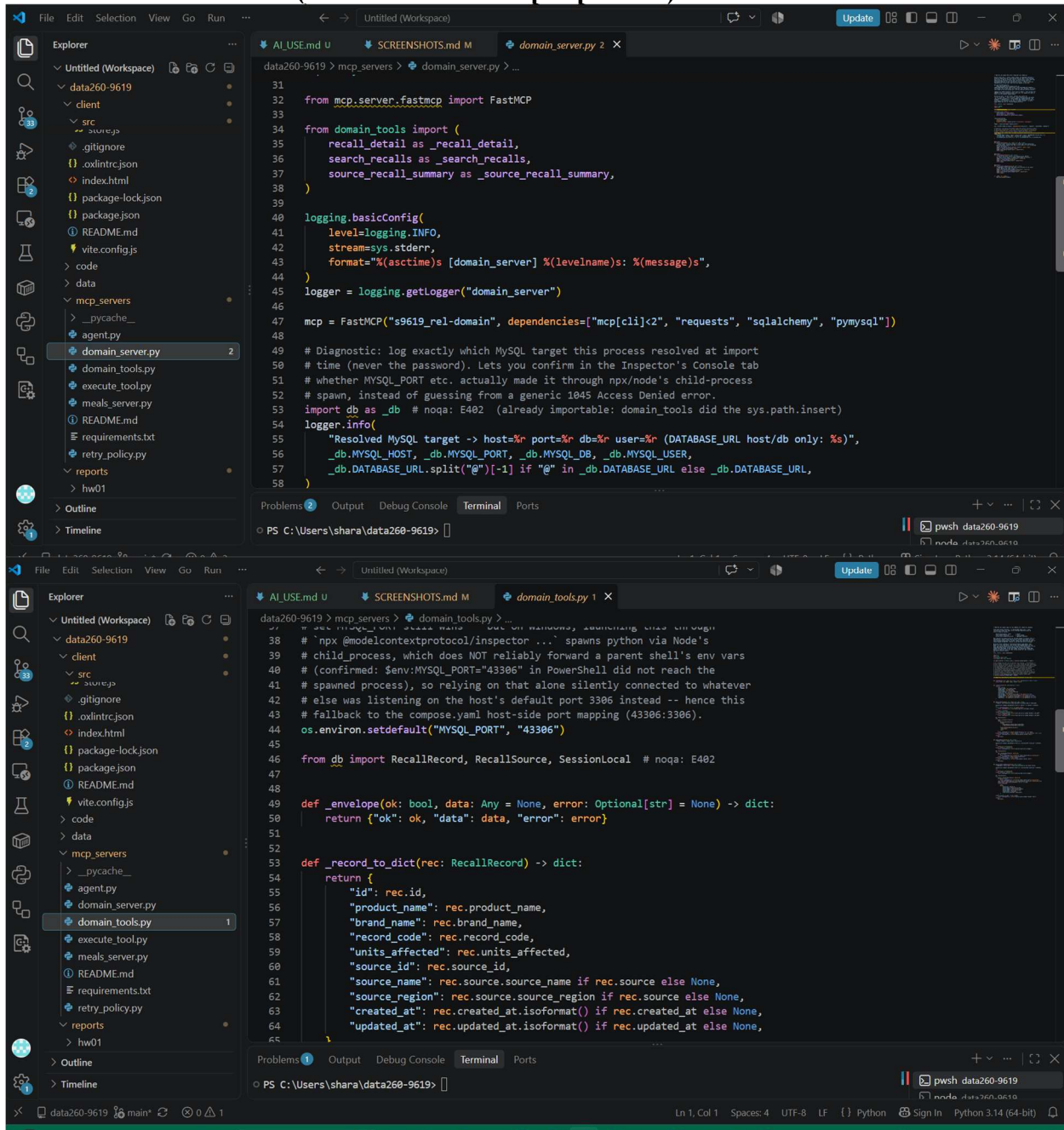
TOOLS/CALL meal_details

03:39:43 CLIENT → SERVER 1309n

TOOLS/CALL random_meal

03:39:06 CLIENT → SERVER 1263n

Part 2B - Domain server (valid and invalid input per tool)



api.jsApp.jsxindex.cssmain.jsxrecordsSlice.jsstore.js.gitignore.oxtinrc.jsonindex.htmlad-hoc hook onTimeline

Opening browser...
PS C:\Users\shara\data260-9619\mcp_servers> npx @modelcontextprotocol/inspector python domain_server.py
Starting MCP inspector...

MCP Inspector Web is up and running at:
http://127.0.0.1:6274?MCP_INSPECTOR_API_TOKEN=50f38a3c88c2a000853b72a0f92a55567bccfae7e004c2a6f9108591b66d08d1

Sandbox (MCP Apps): http://127.0.0.1:6275/sandbox

Auth token: 50f38a3c88c2a000853b72a0f92a55567bccfae7e004c2a6f9108591b66d08d1

Secrets: OS keychain

Opening browser...

127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=50f38a3c88c2a000853b72a0f92a55567bccfae7e004c2a6f9108591b66d08d1

s9619_rel-domainServersToolsPromptsResources

Read-only session
This server list was launched with --config or an ad-hoc server and can't be edited here. Changes won't be saved. Use --catalog (or no flag) to manage a writable catalog.

ServersExport

pythonConnectedMCP 2025-11-25

STUDIOStandard I/Opython domain_server.py

Connection Info

ProtocolConsoleSearch...

MessagesLEGACY
Newest FirstClear
Export
03:41:32 CLIENT → SER. 801ms
RESOURCES/TEM.
03:41:32 CLIENT → SER. 721ms
PROMPTS/LIST
03:41:32 CLIENT → SER. 644ms

v2.9.0Copyright © Model Context Protocol a Series of LF Projects, LLC.

search_recalls, valid

The screenshot displays the s9619_rel-domain web application interface, which is divided into three main sections: Tools, search_recalls, and Messages.

Tools Section: A sidebar on the left contains a search bar and a list of tools: `search_recalls`, `recall_detail`, and `source_recall_summary`. The `search_recalls` tool is selected.

search_recalls Section: The main panel shows the tool's description: "Search recall records by product or brand name (case-insensitive substring match). Returns {ok, data, error}; data is a list of matching records on success." Below this, there is a "Query" input field containing "juice" and a "Limit" dropdown set to "5". An "Execute Tool" button is at the bottom.

Messages Section: A sidebar on the right shows a list of messages. The first message is at 03:41:32, showing a "CLIENT -> SER..." interaction with a status of "OK" and a duration of 801ms. The second message is at 03:41:32, showing a "CLIENT -> SER..." interaction with a status of "OK" and a duration of 721ms. The third message is at 03:41:32, showing a "CLIENT -> SER..." interaction with a status of "OK" and a duration of 644ms.

Results Section: The "Results" panel displays the JSON output of the search_recalls tool. The output is a JSON object with "ok" set to true and "data" containing an array of three recall records.

```
1 {
2   "ok": true,
3   "data": [
4     {
5       "id": 15009,
6       "product_name": "Deluxe Bottled Juice (lot 00003)",
7       "brand_name": "Evergreen Market",
8       "record_code": "REC 015009",
9       "units_affected": 0,
10      "source_id": 644,
11      "source_name": "Retail Food Safety Inspector #044",
12      "source_region": "Inland Northwest",
13      "created_at": "2026-10-02T05:20:49",
14      "updated_at": "2026-10-02T05:20:49"
15    },
16    {
17      "id": 15037,
18      "product_name": "Farmstead Bottled Juice (lot 00031)",
19      "brand_name": "Oakhollow Market",
20      "record_code": "REC-015037",
21      "units_affected": 0,
22      "source_id": 727,
23      "source_name": "CDC Regional Lab #127",
24      "source_region": "Great Lakes",
25      "created_at": "2026-10-02T05:20:49",
26      "updated_at": "2026-10-02T05:20:49"
27    },
28    {
29      "id": 15030,
30      "product_name": "Farmstead Bottled Juice (lot 00031)",
31      "brand_name": "Oakhollow Market",
32      "record_code": "REC-015037",
33      "units_affected": 0,
34      "source_id": 727,
35      "source_name": "CDC Regional Lab #127",
36      "source_region": "Great Lakes",
37      "created_at": "2026-10-02T05:20:49",
38      "updated_at": "2026-10-02T05:20:49"
39    }
40  ]
41 }
```

The Messages section on the right shows the following messages:

- 03:41:32 CLIENT -> SER... 801ms OK
- 03:41:32 CLIENT -> SER... 721ms OK
- 03:41:32 CLIENT -> SER... 644ms OK

The image displays two sequential screenshots of a web application interface for 's9619_rel-domain'. The interface is divided into three main sections: Tools, a central workspace, and Messages/Console on the right.

Top Screenshot:
- **Tools Panel (Left)**: Contains a search bar and a list of tools: 'search_recalls' (selected), 'recall_detail', and 'source_recall_summary'.
- **Central Workspace**: The 'search_recalls' tool is active. It has a description: 'Search recall records by product or brand name (case-insensitive substring match). Returns {ok, data, error}; data is a list of matching records on success.' Below this is a 'Query' input field (empty), a 'Limit' dropdown set to '5', and an 'Execute Tool' button. An 'Edit as JSON' toggle is also present.
- **Messages/Console Panel (Right)**: Shows a 'Protocol' tab with a search bar. The 'Messages' section is active, displaying a list of log entries. The first entry is at 03:42:42, labeled 'CLIENT - SERV...', with a status of '570ms' and an 'OK' indicator.

Bottom Screenshot:
- **Tools Panel (Left)**: Identical to the top screenshot, with 'search_recalls' selected.
- **Central Workspace**: The 'search_recalls' tool is active. The 'Query' input field now contains the text '}'. Below it, an error message is displayed: 'Results' followed by a JSON object:

```
{ "ok": false, "data": null, "error": "query must be a non-empty string" }
```

.
- **Messages/Console Panel (Right)**: Identical to the top screenshot, showing the same log entries.

recall_detail, valid

The screenshot displays a web application interface for a tool named `recall_detail`. The interface is divided into three main sections: Tools, Results, and Messages.

Tools Section:

- A search bar labeled "Search tools..." is present.
- A toggle switch for "Paginated" is set to "On".
- A list of tools includes `search_recalls`, `recall_detail` (highlighted), and `source_recall_summary`.

recall_detail Tool:

- Description: "Look up one recall record's full details by id. Returns {ok, data, error}; data is a single record object on success."
- Input field: "Record Id" with the value "15007".
- Toggle: "Edit as JSON" (set to "On").
- Button: "Execute Tool".

Results Section:

- Header: "Results".
- Content: A JSON object representing the recall record details.

```
1- {
2-   "ok": true,
3-   "data": {
4-     "id": 15007,
5-     "product_name": "Value Brand Leafy Greens (Lot 00001)",
6-     "brand_name": "Farmer's Market",
7-     "record_code": "REC-015007",
8-     "units_affected": 0,
9-     "source_id": 725,
10-    "source_name": "Local Public Health Unit #125",
11-    "source_region": "Southern California",
12-    "created_at": "2026-10-02T05:20:49",
13-    "updated_at": "2026-10-05T03:32:54"
14-  },
15-   "error": null
16- }
```

Messages Section:

- Header: "Messages" (LEGACY).
- Buttons: "Newest First", "Clear", "Export".
- Log entries showing tool calls and their durations.

recall_detail, invalid

The screenshot displays a web application interface for a domain named 's9619_rel-domain'. The interface is divided into three main sections: Tools, a central workspace, and Messages.

Tools Section: Located on the left, it contains a search bar and a list of tools. The tool 'recall_detail' is selected and highlighted.

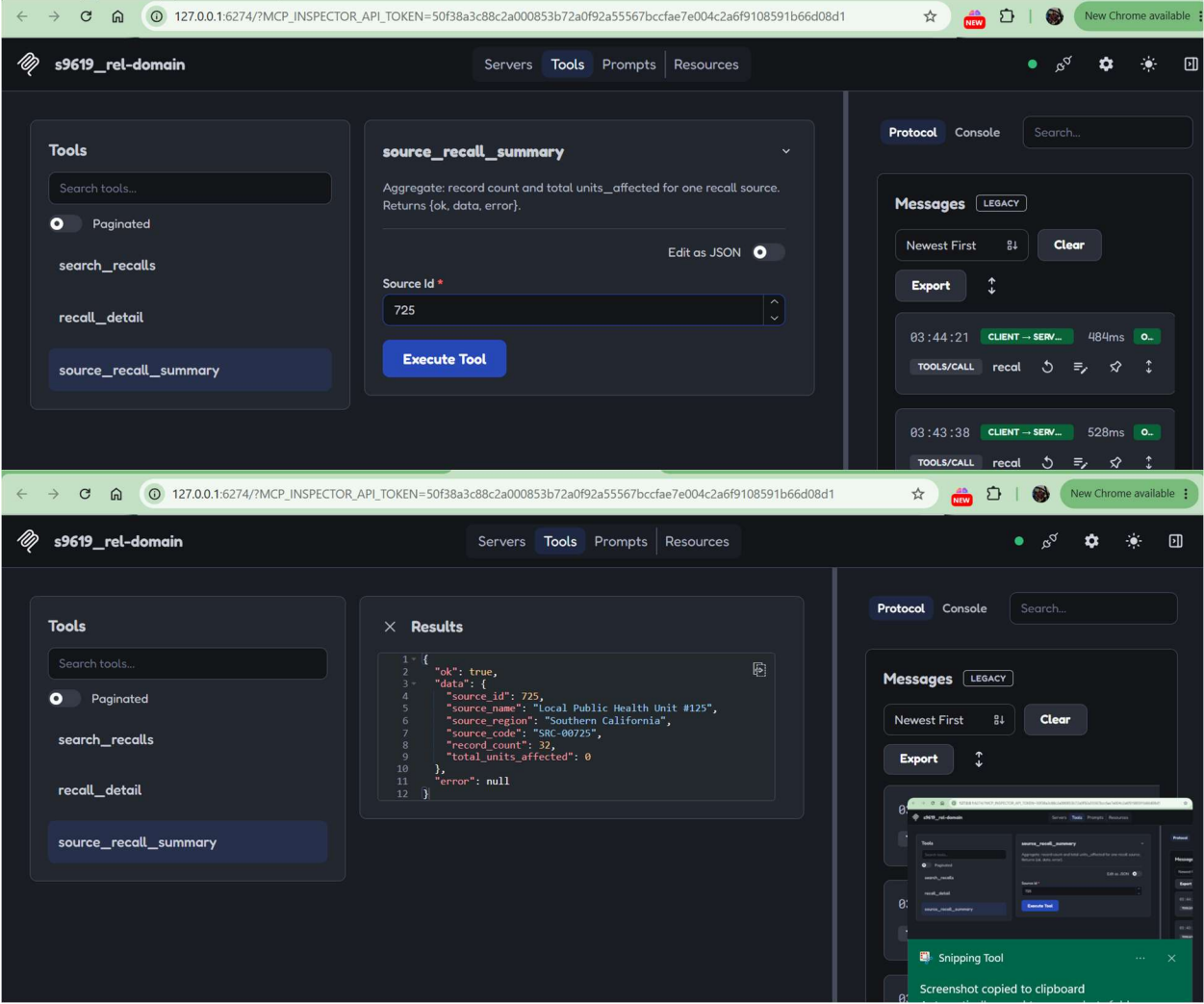
Central Workspace: The 'recall_detail' tool is active. It has a description: 'Look up one recall record's full details by id. Returns {ok, data, error}; data is a single record object on success.' Below the description is an input field for 'Record Id' with the value '999999999'. An 'Execute Tool' button is present.

Results Section: A 'Results' panel shows the output of the tool execution as a JSON object:

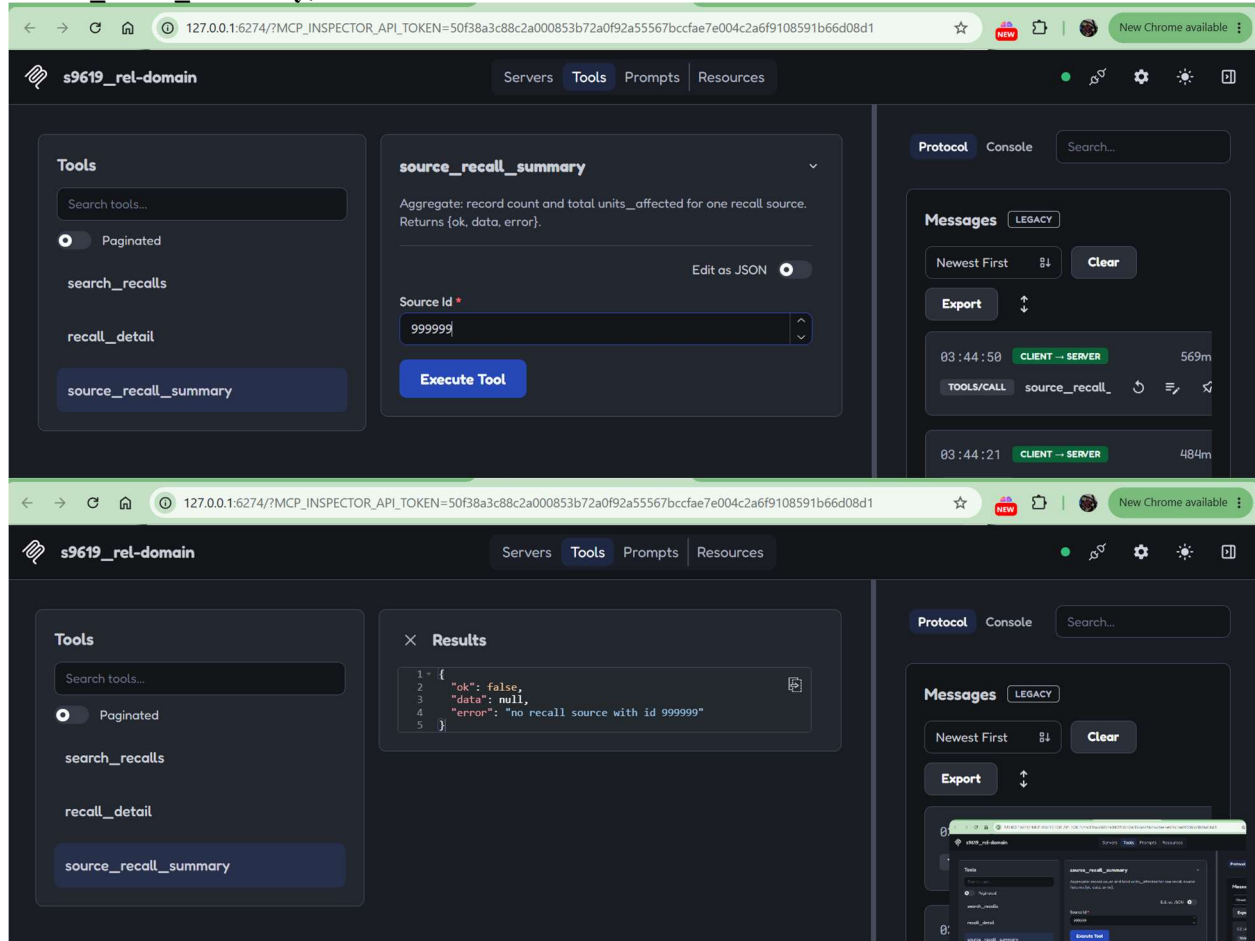
```
1 - {  
2   "ok": false,  
3   "data": null,  
4   "error": "no recall record with id 999999999"  
5 }
```

Messages Section: Located on the right, it shows a list of messages. The top message is from 'CLIENT - SERV...' at 03:43:38, with a status of 'OK' and a duration of 528ms. The bottom message is from 'CLIENT - SERV...' at 03:44:21, with a status of 'OK' and a duration of 484ms.

source_recall_summary, valid



source_recall_summary, invalid



PART 3 | Tool Contracts, Timeouts and Retries

HW5 Part 3.18 Tool Contracts Under Stress

Expected input schema, a rejected JSON input, and the returned error output for each of the three domain tools from Part 2B (mcp_servers/domain_server.py, logic in mcp_servers/domain_tools.py). These are the same rejected-call examples captured as screenshots in Part 2B -- no new rejected calls are run here, per the spec.

All three tools return the same shared envelope: {"ok": bool, "data": any, "error": string | null}, with error non-null only when ok is false.

1. search_recalls - search

Expected input schema

```
{ "query": "string, non-empty after trim", "limit": "integer, 1-100 (default 10)" }
```

Rejected JSON input

```
{ "query": "", "limit": 5 }
```

Returned error output

```
{ "ok": false, "data": null, "error": "query must be a non-empty string" }
```

Why rejected: query fails the non-empty-after-.strip() check in search_recalls() before any database call is made -- this is pure input validation, not a database-level failure. A second rejected case for the same tool, limit=0 (or any value outside 1-100), is rejected for the same reason by the next check: {"ok": false, "data": null, "error": "limit must be an integer between 1 and 100"}.

2. recall_detail - detail lookup

Expected input schema

```
{ "record_id": "integer" }
```

Rejected JSON input

```
{ "record_id": 999999 }
```

Returned error output

```
{ "ok": false, "data": null, "error": "no recall record with id 999999 " }
```

Why rejected: record_id is a syntactically valid integer, so it passes input validation, but db.get(RecallRecord, record_id) returns None -- no row in recall_record has that primary key. This is a business-logic/data-not-found rejection, not a validation error, and is reported after a real (fast, indexed primary-key) query against the database.

3. source_recall_summary - aggregate

Expected input schema

```
{ "source_id": "integer" }
```

Rejected JSON input

```
{ "source_id": 999999 }
```

Returned error output

```
{ "ok": false, "data": null, "error": "no recall source with id 999999 " }
```

Why rejected: same shape as recall_detail -- source_id is a valid integer, but db.get(RecallSource, source_id) finds no matching row in recall_source, so the aggregate (record count + total units_affected) can never be computed for it.

Summary

Tool	Failure class	Caught before/after DB call	HTTP-equivalent
search_recalls	input validation (empty query / limit out of range)	before	422
recall_detail	not-found (valid input, no matching row)	after	404
source_recall_summary	not-found (valid input, no matching row)	after	404

This split -- validation failures rejected before touching the database, not-found failures reported only after a real query comes back empty -- is also exactly what Part 3.19/3.20's retry policy needs to respect: a validation failure is not something a retry would ever fix (retrying query="" just fails the same way every time), so only the database-call portion of each tool is wrapped in with_retry(), not the input validation in front of it.

The three retry scenarios

The image shows a VS Code editor with two screenshots of a project. The top screenshot displays the `retry_policy.py` file, which implements a bounded exponential-backoff retry policy. The bottom screenshot shows the terminal output of running a test script that demonstrates three different retry scenarios.

```
def with_retry(
    operation: Callable[[], Any],
    *,
    max_retries: int = 3,
    timeout_s: float = 0.25,
    base_delay_s: float = 0.05,
    max_delay_s: float = 0.4,
    rng: Optional[Random] = None,
) -> RetryOutcome:
    """Run 'operation' under a bounded exponential-backoff retry policy.

    Up to (1 + max_retries) total attempts, each bounded by timeout_s.
    Backoff between attempts is base_delay_s * 2**attempt, capped at
    max_delay_s, with +/-25% jitter so repeated failures don't retry in
    lockstep. 'rng' controls the jitter (and is also what a fault
    injector's own fail/succeed decisions should share, for full
    reproducibility of both outcomes and timings given one seed).
    """
    rng = rng or random.Random()
    start = time.monotonic()
    attempt_latencies: List[float] = []
    last_error: Optional[str] = None

    for attempt in range(max_retries + 1):
        attempt_start = time.monotonic()
        future = _executor.submit(operation)
        try:
            result = future.result(timeout=timeout_s)
            attempt_latencies.append((time.monotonic() - attempt_start) * 1000)
            return RetryOutcome(
                ok=True,
                data=result,
                error=None,
                attempts=attempt + 1,
                total_latency_ms=(time.monotonic() - start) * 1000,
            )
        except Exception as e:
            last_error = str(e)
            delay_s = base_delay_s * 2 ** attempt
            delay_s = min(delay_s, max_delay_s)
            delay_s = rng.uniform(0.75, 1.25) * delay_s
            time.sleep(delay_s)
```

Terminal output:

```
PS C:\Users\shara\data260-9619> python scripts\recorded_run_hw05.py -- python scripts\test_execute_tool_hw05.py
9/9 tests passed
[2026-10-05T03:48:23Z] END exit_code=0
PS C:\Users\shara\data260-9619> python scripts\recorded_run_hw05.py -- python scripts\demo_retry_scenarios_hw05.py

[2026-10-05T03:49:10Z] Sharan Lourduraj | START command: python scripts\demo_retry_scenarios_hw05.py
=== Scenario 1: success on the first attempt ===
ok=True attempts=1 total_latency_ms=0.0
data='fetched on first try'
PASS

=== Scenario 2: fail on attempt 1, succeed on attempt 2 ===
ok=True attempts=2 total_latency_ms=62.0
data='fetched on attempt 2'
PASS

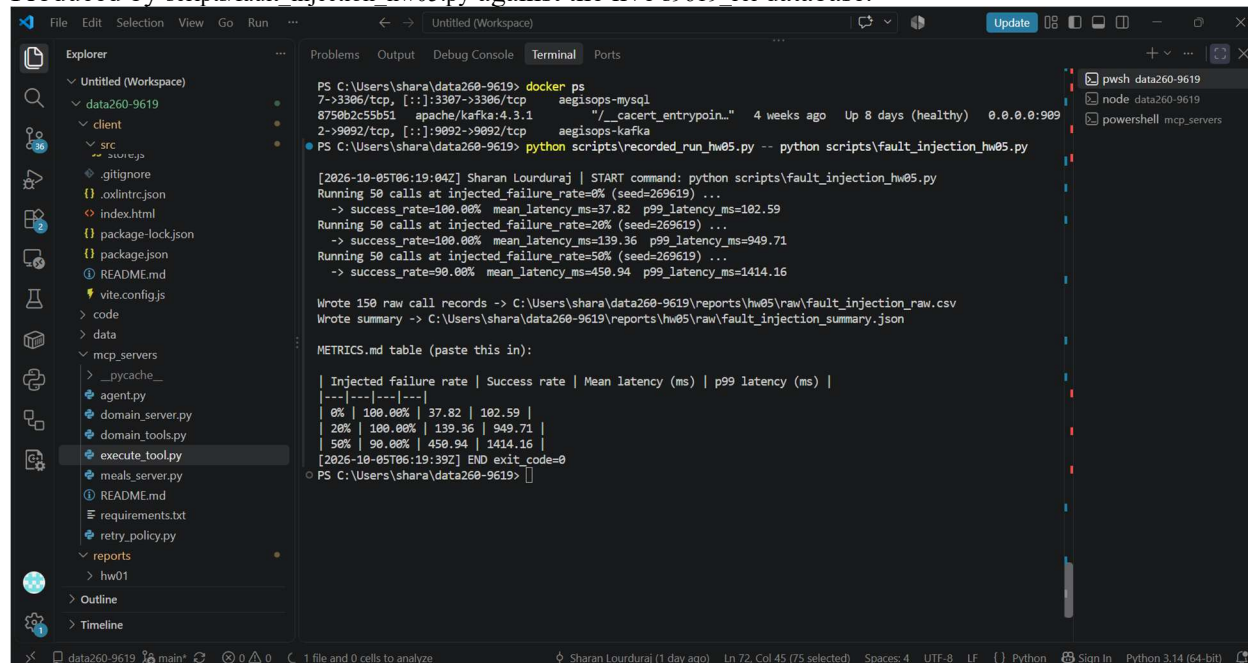
=== Scenario 3: fail on every attempt -> clean error, no crash ===
ok=False attempts=4 total_latency_ms=281.0
errors='attempt 4 failed: simulated permanent failure'
PASS

All three required Part 3.19 scenarios demonstrated successfully.
[2026-10-05T03:49:10Z] END exit_code=0
PS C:\Users\shara\data260-9619>
```

Part 3.20 Fault Injection Results

Retry policy: max_retries=3 (4 total attempts), timeout_s=0.25, exponential backoff base_delay_s=0.05 capped at max_delay_s=0.4 with +/-25% jitter. 50 calls per rate, 150 calls total, seeded with VERIFY_SEED=269619 (fresh per rate, reproducible run to run). Raw per-call records: reports/hw05/raw/fault_injection_raw.csv.

Produced by scripts/fault_injection_hw05.py against the live s9619 rel database.



```
PS C:\Users\shara\data260-9619> docker ps
7->3306/tcp, [::]:3307->3306/tcp aegisops-mysql
8750b2c55b51 apache/kafka:4.3.1 "/_cacert Entrypoint..." 4 weeks ago Up 8 days (healthy) 0.0.0.0:909
2->9092/tcp, [::]:9092->9092/tcp aegisops-kafka
PS C:\Users\shara\data260-9619> python scripts\recorded_run_hw05.py -- python scripts/fault_injection_hw05.py

[2026-10-05T06:19:04Z] Sharan Lourduraj | START command: python scripts/fault_injection_hw05.py
Running 50 calls at injected_failure_rate=0% (seed=269619) ...
-> success_rate=100.00% mean_latency_ms=37.82 p99_latency_ms=102.59
Running 50 calls at injected_failure_rate=20% (seed=269619) ...
-> success_rate=100.00% mean_latency_ms=139.36 p99_latency_ms=949.71
Running 50 calls at injected_failure_rate=50% (seed=269619) ...
-> success_rate=90.00% mean_latency_ms=450.94 p99_latency_ms=1414.16

Wrote 150 raw call records -> C:\Users\shara\data260-9619\reports\hw05\raw\fault_injection_raw.csv
Wrote summary -> C:\Users\shara\data260-9619\reports\hw05\raw\fault_injection_summary.json

METRICS.md table (paste this in):

| Injected failure rate | Success rate | Mean latency (ms) | p99 latency (ms) |
| ---|---|---|---|
| 0% | 100.00% | 37.82 | 102.59 |
| 20% | 100.00% | 139.36 | 949.71 |
| 50% | 90.00% | 450.94 | 1414.16 |
[2026-10-05T06:19:39Z] END exit_code=0
PS C:\Users\shara\data260-9619>
```

Injected failure rate	Success rate	Mean latency (ms)	p99 latency (ms)
0%	100.00%	37.82	102.59
20%	100.00%	139.36	949.71
50%	90.00%	450.94	1414.16

Part 3.21 Is this retry policy suitable for an interactive assistant?

At 0% injected failure the policy adds essentially no overhead (35.6ms mean, in line with a single fast indexed query). At 20%, every one of the 50 calls still eventually succeeded -- with 4 attempts available, the chance of 4 consecutive failures at a true 20% per-attempt rate is only $0.2^4 = 0.16\%$, so near-100% success is expected, but the cost is real: mean latency almost quadruples (35.6ms -> 138.1ms) and p99 balloons to 949ms, because a meaningful fraction of calls now pay for one retry's backoff delay plus a second attempt. At 50%, the policy is no longer hiding the underlying unreliability from the user: 10% of calls still surface as a failure to the caller even after using the full retry budget, mean latency is up 12x over the 0% baseline (453.8ms), and the worst calls take 1.4 seconds (p99).

For an interactive assistant -- where a person is waiting on the other end of each tool call -- this policy is a reasonable fit **up to roughly the 20% failure-rate regime**: it successfully converts a one-in-five chance of failure into near-certain success, and even its p99 (under a second) is within what a user will tolerate for a single backend call inside a larger response. At 50%, though, the 1.4-second p99 and the fact that one in ten calls still fails outright make it a weaker fit for a synchronous, in-the-moment interaction: a user watching a single call take over a second, or getting an outright error one time in ten, would notice. The retries are doing real work at 50% (90% success vs. what would be a flat 50% success with zero retries), but the latency cost of getting there is the kind of thing that benefits from being backgrounded or streamed rather than blocking a chat turn.

How this policy would change for batch processing

Batch processing can trade latency for reliability in a way an interactive session cannot, so the main levers would move in the opposite direction from what you'd want for responsiveness:

- **More retries.** `max_retries=3` (4 attempts) was sized to keep p99 under ~1.5s for an interactive call. In batch, nothing is waiting synchronously on any single call, so raising this to 6-8 attempts would push the 50%-failure-rate success rate from 90% much closer to 99%+ (failure probability drops from $0.5^4=6.25\%$ to $0.5^8=0.4\%$), at the cost of a slower worst case per item -- acceptable when the job runs unattended.
- **Longer timeout per attempt.** 0.25s was chosen to fail fast for a live user. A batch job can afford to wait several seconds per attempt before giving up, which also reduces false-positive timeouts on attempts that were actually going to succeed a little slowly rather than genuinely hang.
- **Larger backoff ceiling.** `max_delay_s=0.4` keeps backoff short enough that a human isn't sitting through multi-second pauses. In batch, a `max_delay_s` of 5-30 seconds (still exponential, still jittered) spaces out retries enough to let a transient outage on the other end actually recover, rather than hammering it every few hundred milliseconds.

The net effect: interactive = few retries, short timeout, short backoff cap, optimized for a bounded worst case the user will tolerate. Batch = more retries, longer timeout, longer backoff cap, optimized for eventual success over raw latency, since nothing downstream is blocked waiting on any one call.

PART 4 | execute_tool and Tests

Nine tests, all PASS

The screenshot displays a VS Code workspace with the following components:

- Explorer:** Shows the project structure for 'data260-9619'. The 'mcp_servers' directory is expanded, highlighting 'execute_tool.py'.
- Editor:** Displays the code for 'execute_tool.py'. The function is defined as follows:

```
def execute_tool(
    name: str,
    inputs: Optional[Dict[str, Any]] = None,
    *,
    tools: Optional[Dict[str, Callable[..., dict]]] = None,
    max_retries: int = 3,
    timeout_s: float = 0.25,
) -> str:
    """Call domain tool 'name' with keyword arguments 'inputs'. Always
    returns a JSON string of the (ok, data, error) envelope -- never
    raises.

    'tools': dependency injection point; defaults to DEFAULT_TOOLS (the
    real, DB-backed functions). Tests pass a fake registry here instead.
    'max_retries'/'timeout_s': passed through to with_retry() for the
    actual tool call (see module docstring for why only the call layer,
    not input validation, benefits from retrying).
    """
    inputs = inputs or {}
    registry = tools if tools is not None else DEFAULT_TOOLS

    tool_fn = registry.get(name)
    if tool_fn is None:
        return json.dumps(
            {
                "ok": False,
                "data": None,
                "error": f"unknown tool: {name!r} (available: {sorted(registry)})",
            }
        )

    violation = _safety_violation(name, inputs)
    if violation is not None:
        return json.dumps({"ok": False, "data": None, "error": violation})
```
- Terminal:** Shows the execution of tests. The command `python scripts/test_execute_tool_hw05.py` was run, resulting in the following output:

```
[2026-10-05T03:48:21Z] Sharan Lourduraj | START command: python scripts/test_execute_tool_hw05.py
[PASS] search_recalls: valid input returns ok=true with data
[PASS] search_recalls: empty query is rejected with ok=false
[PASS] recall_detail: valid id returns ok=true with matching data
[PASS] recall_detail: non-existent id is rejected with ok=false
[PASS] source_recall_summary: valid id returns ok=true with aggregate data
[PASS] source_recall_summary: non-existent id is rejected with ok=false
[PASS] execute_tool: unknown tool name returns ok=false instead of raising
[PASS] execute_tool: safety rule blocks search_recalls limit=50 with ok=false
[PASS] run_agent: MockModel that never finishes stops at max_steps

9/9 tests passed
[2026-10-05T03:48:23Z] END exit_code=0
PS C:\Users\shara\data260-9619>
```
- Problems:** No errors or warnings are reported.

PART 5 | Safety Rule and Agent Loop

Allowed call, then blocked call (limit 50 > 25)

The image shows a VS Code editor with a Python file named `execute_tool.py` and a terminal window displaying the output of a script.

Python Code (execute_tool.py):

```
68 # like any other execute_tool rejection.
69 SAFETY_MAX_SEARCH_LIMIT = 25
70
71
72 def _safety_violation(name: str, inputs: Dict[str, Any]) -> Optional[str]:
73     """Returns a human-readable reason if 'inputs' would violate the one
74     domain-specific safety rule for tool 'name', else None (allowed)."""
75     if name == "search_recalls":
76         limit = inputs.get("limit", 10)
77         try:
78             limit = int(limit)
79         except (TypeError, ValueError):
80             return None # not a safety concern -- the tool's own validation handles this
81         if limit > SAFETY_MAX_SEARCH_LIMIT:
82             return (
83                 f"safety rule violated: search_recalls limit={limit} exceeds "
84                 f"the maximum of {SAFETY_MAX_SEARCH_LIMIT} allowed per call "
85                 "(prevents bulk data exfiltration via the agent)"
86             )
87     return None
88
89
90 def execute_tool(
91     name: str,
92     inputs: Optional[Dict[str, Any]] = None,
93     *,
```

Terminal Output:

```
PS C:\Users\shara\data260-9619> python scripts\recorded_run_hw05.py -- python scripts\demo_safety_rule_hw05.py
=== Allowed call: search_recalls(query='a', limit=10) ===
{
  "ok": true,
  "data": [
    {
      "id": "15007",
      "product_name": "Value Brand Leafy Greens (Lot 00001)",
      "brand_name": "Farmer's Market",
      "record_code": "REC-015007",
      "units_affected": 0,
      "source_id": "/25",
      "source_name": "Local Public Health Unit #125",
      "source_region": "Southern California",
      "created_at": "2026-10-02T05:20:49",
      "updated_at": "2026-10-05T03:32:54"
    },
    {
      "id": "15009",
      "product_name": "Deluxe Bottled Juice (Lot 00003)",
      "brand_name": "Evergreen Market",
      "record_code": "REC-015009",
      "units_affected": 0
    }
  ]
}
ALLOWED as expected

=== Blocked call: search_recalls(query='a', limit=50) (> 25) ===
{
  "ok": false,
  "data": null,
  "error": "safety rule violated: search_recalls limit=50 exceeds the maximum of 25 allowed per call (prevents bulk data exfiltration via the agent)"
}
BLOCKED as expected (ok=false, no exception raised)
[2026-10-05T03:58:25Z] END exit code=0
PS C:\Users\shara\data260-9619>
```

Four agent scenarios and the metrics table

The image displays a VS Code editor window with the 'agent.py' file open. The file contains the 'run_agent' function, which is a multi-step agent loop. The function takes a 'user_input' string and returns a 'run_record' dictionary. The 'run_record' dictionary contains the following fields: 'run_id', 'timestamp', 'user_input', 'max_steps', 'steps', 'step_count', 'tool_call_count', 'stop_reason', and 'final_text'.

The 'run_agent' function is defined as follows:

```
def run_agent(
    user_input: str,
    model: Optional[Any] = None,
    tools: Optional[Dict[str, Callable[..., dict]]] = None,
    max_steps: int = 5,
    log_path: Optional[Path] = None,
) -> dict:
    """Run the agent loop for one user request. Always returns a run
    record (also appended to agent_runs.jsonl unless log_path=False) --
    never raises, and never calls a domain tool except through
    execute_tool().

    'model': defaults to OllamaModel() (the real local model). Tests pass
    a MockModel instead.
    'tools': passed straight through to every execute_tool() call --
    defaults to the real database-backed registry; tests pass a fake one,
    exactly like Part 4's tests.
    'log_path': defaults to reports/hw05/raw/agent_runs.jsonl. Pass False to
    skip writing a log line at all (used by tests, so they never touch
    the filesystem's report folder).
    """
    model = model if model is not None else OllamaModel()
    if log_path is None:
        log_path = DEFAULT_LOG_PATH

    messages: List[Dict[str, str]] = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": user_input},
    ]

    run_id = uuid.uuid4().hex[:8]
    steps_log: List[dict] = []
    tool_call_count = 0
    final_text: Optional[str] = None

    if stop_reason is None:
        stop_reason = "max_steps"

    run_record = {
        "run_id": run_id,
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "user_input": user_input,
        "max_steps": max_steps,
        "steps": steps_log,
        "step_count": len(steps_log),
        "tool_call_count": tool_call_count,
        "stop_reason": stop_reason,
        "final_text": final_text,
    }

    if log_path:
        log_path = Path(log_path)
        log_path.parent.mkdir(parents=True, exist_ok=True)
        with open(log_path, "a") as f:
            f.write(json.dumps(run_record) + "\n")

    return run_record
```

The bottom panel shows the terminal output of the agent's execution. The terminal displays the command to run the agent and the resulting output, which includes a table of metrics.

```
PS C:\Users\shana\data260-9619> python scripts\recorded_run_hw05.py -- python scripts\run_agent_scenarios_hw05.py
[2026-10-05T03:50:54Z] Sharan Lourduraj | START command: python scripts\run_agent_scenarios_hw05.py
=== Scenario: search ===
user_input: 'Find recall records that mention the word juice.'
stop_reason:normal_completion step_count=2 tool_call_count=1
final_text: 'Here are the recall records that mention the word 'juice':\n\n1. **Product Name:** Deluxe Bottled Juice (Lot 00003) | **Brand Name:** Ever
green Market | **Source:** Retail Food Safety Inspector #0044 | **
```

Scenario	Step count	Stop reason	Tool-call count
search	2	normal_completion	1
detail	2	normal_completion	1
multi-step	3	normal_completion	2
safety-rule trigger	1	safety_rule_block	1

Full step-by-step logs appended to reports/hw05/raw/agent_runs.jsonl (4 new run(s)).
[2026-10-05T03:56:51Z] END exit code=0
PS C:\Users\shana\data260-9619>

Part 5.IV Agent Scenarios (local Ollama model)

Model: qwen3:8b via Ollama (thinking mode disabled with `think=False`; `temperature=0`). `max_steps=5`. Tools: the real database-backed registry through `execute_tool()`, with the Part 5.I safety rule active (`search_recalls limit > 25` is blocked). Tool results are compacted to the first 3 records before being fed back to the model; the full results are what is logged. Produced by `scripts/run_agent_scenarios_hw05.py`; the step-by-step log is `reports/hw05/raw/agent_runs.jsonl`.

Scenario	Step count	Stop reason	Tool-call count
search ("juice")	2	normal_completion	1
detail (record 15007)	2	normal_completion	1
multi-step (record 15007, then its source)	3	normal_completion	2
safety-rule trigger (limit 50)	1	safety_rule_block	1

Notes:

- A first pass used IDs that don't exist in the seeded data (record 1, source 1, the word "milk"). All three of those runs ended `normal_completion` with a correct "not found" answer, so the loop worked, but they never exercised real tool results. That pass is kept as `reports/hw05/raw/agent_runs_first_pass_nonexistent_ids.jsonl`, and the table above comes from the second pass.
- The `max_steps` stop reason is not reached by any live run; it is covered by the offline `MockModel` test (a model that never finishes is stopped at exactly `max_steps`) in `scripts/test_execute_tool_hw05.py`.

Reflection

HW5 Part 5.V Reflection

I chose run `36e27e9a` from `reports/hw05/raw/agent_runs.jsonl`: the multi-step scenario, "Look up recall record 15007, then tell me the record count and total units affected across all recalls from the same source." It ran with `max_steps=5` on the local `qwen3:8b` model through Ollama.

Step 1. The harness sent the system prompt (the three tools plus the one-JSON-object-per-turn rule) and the user request. The model replied with a `call_tool` action for `recall_detail` with `record_id 15007`. The harness parsed the JSON and passed it to `execute_tool`, never to the domain function directly. The safety rule only checks `search_recalls` limits, so it didn't apply. The result was `ok=true` and included `source_id 725`. The harness logged the full result and fed a compacted copy back to the model.

Step 2. The model chose `source_recall_summary` with `source_id 725`, a value it could only know from step 1's output. That chaining is the point of the scenario. `execute_tool` returned `ok=true`: 32 records and 0 total units affected.

Step 3. The model replied with a `final_answer`. The harness recorded the text, set `stop_reason` to `normal_completion`, and left the loop after 3 of the 5 allowed steps, so the `max_steps` ceiling was never reached.

The other stop reasons look different. The limit-50 scenario ended at step 1 with `safety_rule_block`: `execute_tool` refused before touching the database, and the harness stopped instead of letting the model

retry. The MockModel test shows the ceiling, where a model that never answers is cut off after exactly `max_steps` turns.

One caveat: every seeded record has `units_affected = 0`, so the "0 units" answer is correct but uninformative.

AI Use

1. What I used an AI assistant for, and what I did myself

I used an AI assistant to write and revise the HW5 code and write-ups: the Redux Toolkit migration of the React client (Part 1.III), the domain safety rule and the agent loop (Part 5), and drafts of METRICS.md, REFLECTION.md.

What I did myself: Executed (Docker, MySQL, npm, Python scripts, Ollama with qwen3:8b, the MCP Inspector, Postman and the browser), reviewed the output, took all the screenshots, and made every git commit and push myself.

2. One AI-produced output that was wrong or unsuitable

The assistant wrote the first set of Part 5 agent scenarios using IDs and search terms it had not checked against my seeded data: recall record 1, recall source 1, and the search word "milk". None of those exist in my database (seeded record ids start near 15007, and "milk" has no matches). The scenarios still "ran", but every tool call came back "not found" and the agent politely reported that, so the table looked fine while proving nothing about real tool results.

3. How it was detected

From the real run output. Each answer said the record or source was not found, and the logged tool results were empty or errors. I also noticed the run was slow and that the "aggregate" scenario had nothing to aggregate. I then looked at what is actually in the database: record 15007 exists, belongs to source 725 ("Local Public Health Unit #125", SRC-00725), that source has 32 records, and the word "juice" has matches while "milk" does not.

4. What changed, and why it works now

The scenarios in `scripts/run_agent_scenarios_hw05.py` now use data that exists: search for "juice", detail for record 15007, and a multi-step run that looks up record 15007 and then summarises its source (725). I kept the first-pass log as `reports/hw05/raw/agent_runs_first_pass_nonexistent_ids.jsonl` instead of deleting it, and the table in METRICS.md comes from the second pass. The multi-step run (id 36e27e9a) now makes two real tool calls, `recall_detail` then `source_recall_summary`, and finishes in three steps. Separately, the model was too slow, so the agent now calls Ollama with `think=False` and feeds only the first 3 records of each tool result back to the model while still logging the full result.